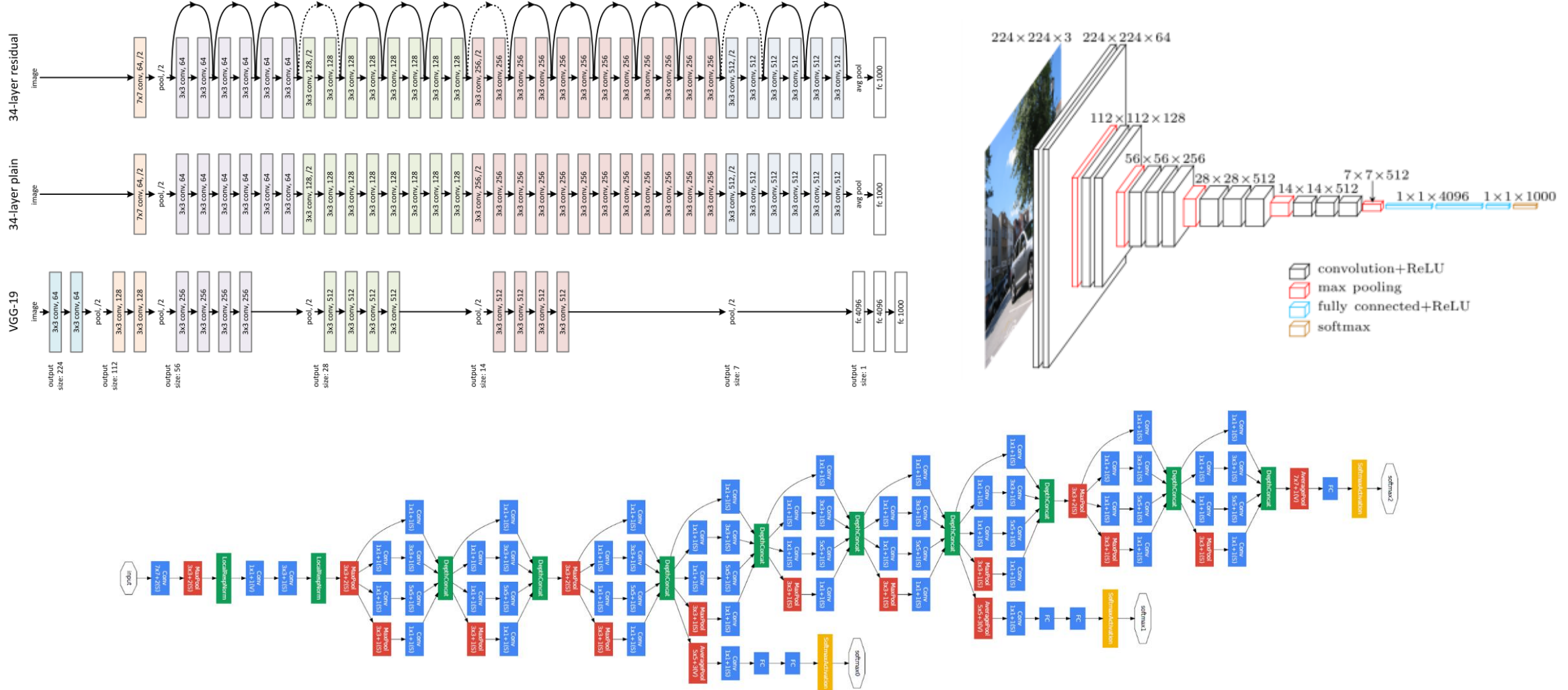


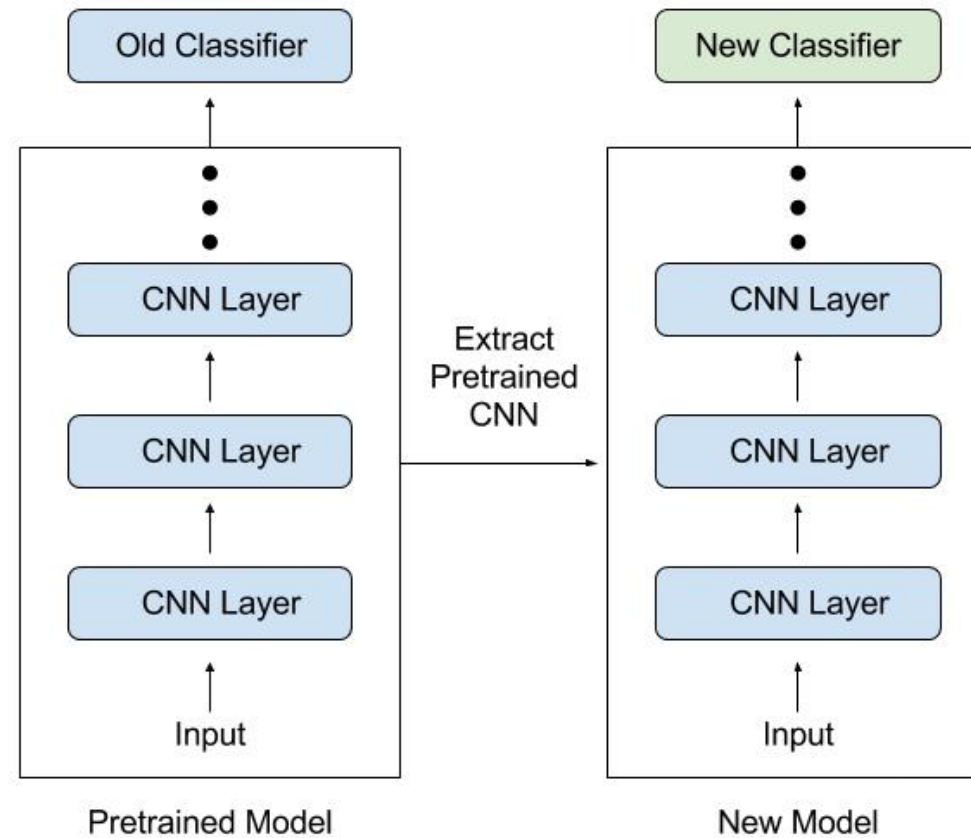
Deep Learning

Lecture 5

Previously, deep CNN architectures for image classification

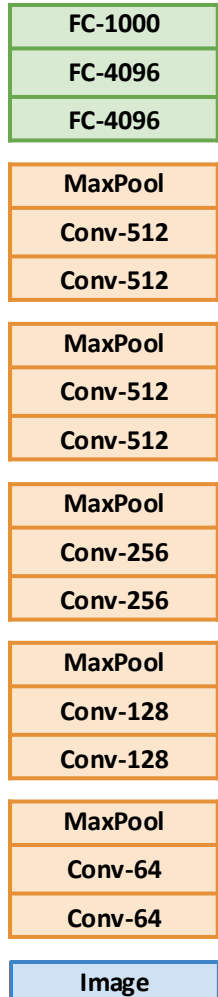


Today: Transfer Learning

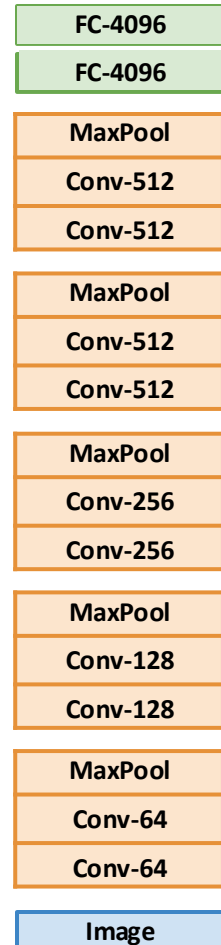


Transfer Learning with CNNs

1. Train on Imagenet



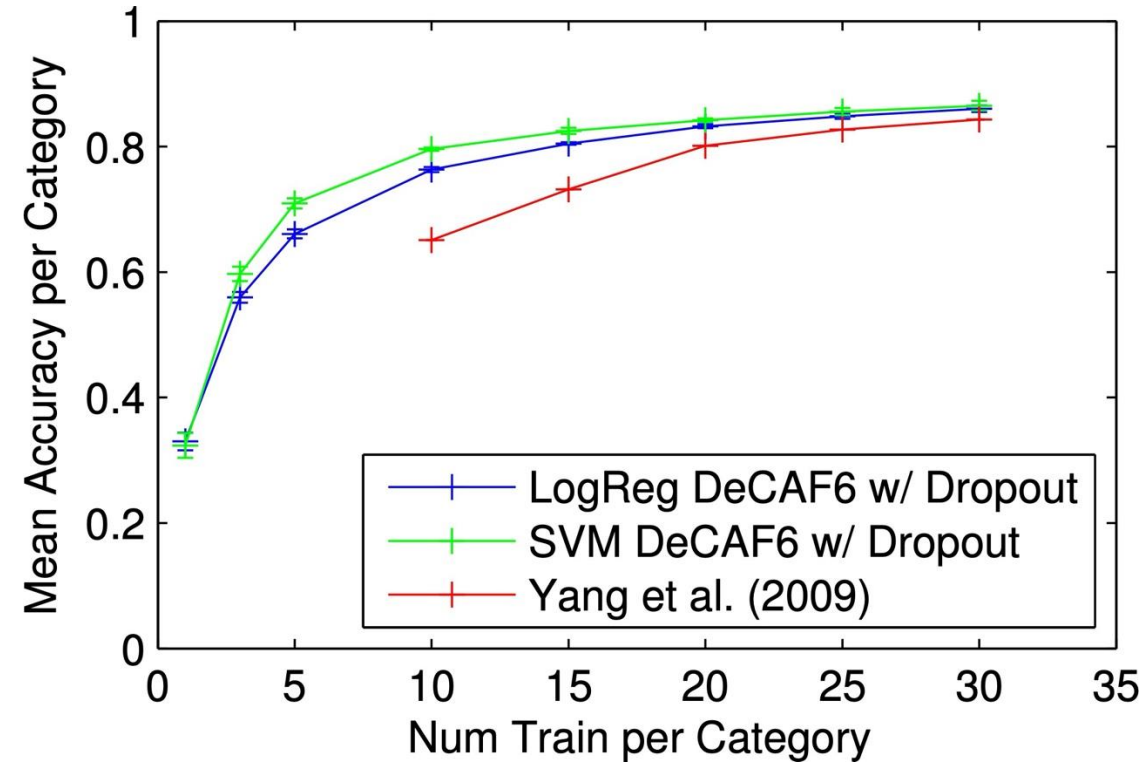
2. Use CNN as a feature extractor



Remove
last layer

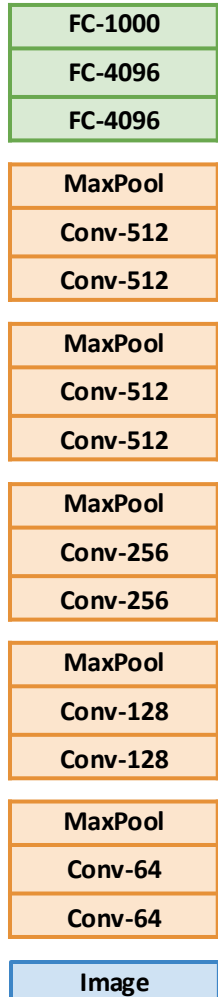
Freeze
these

Classification on Caltech-101

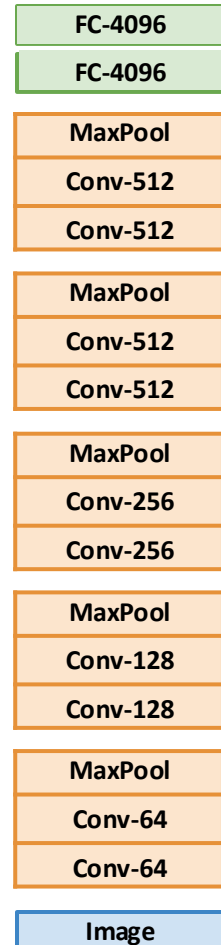


Transfer Learning with CNNs

1. Train on Imagenet



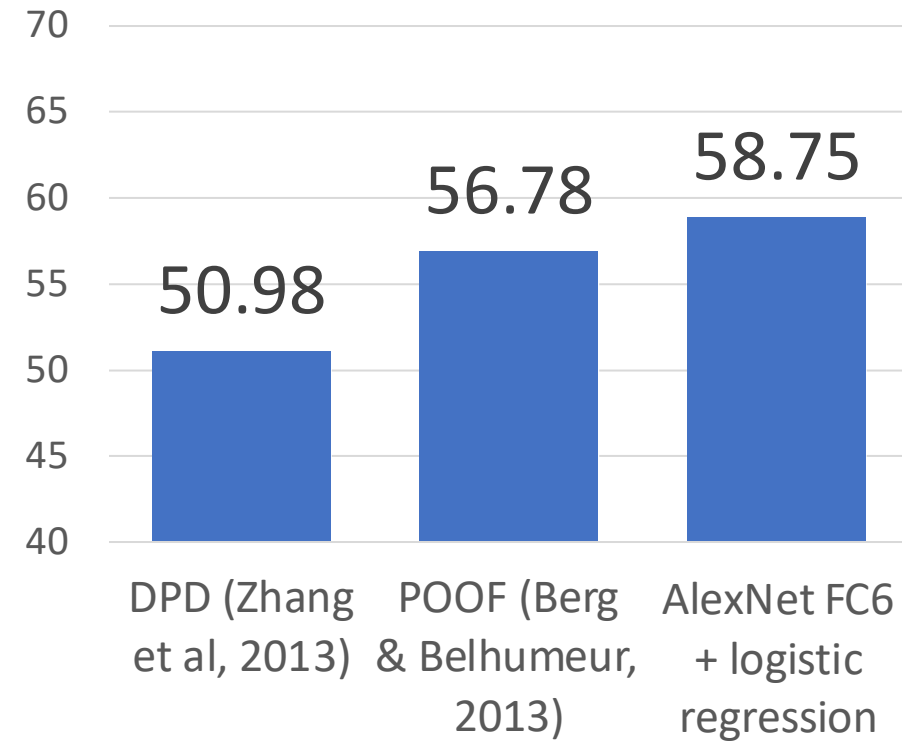
2. Use CNN as a feature extractor



Remove
last layer

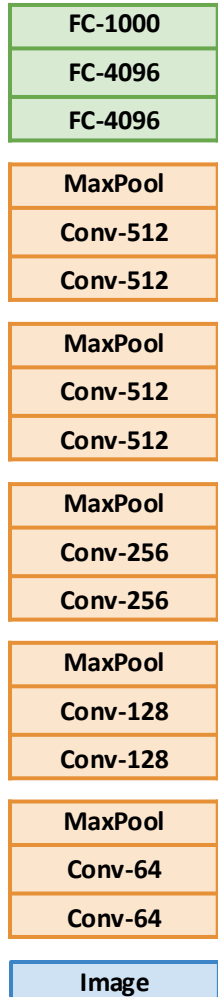
Freeze
these

Bird Classification on Caltech-UCSD

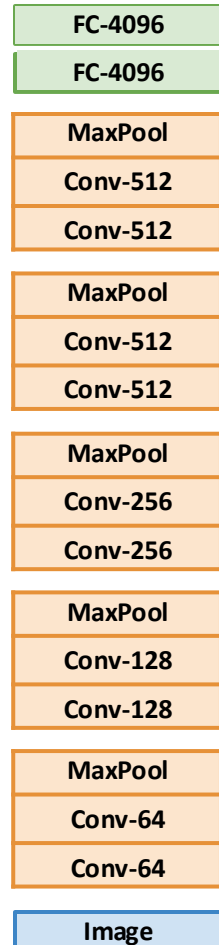


Transfer Learning with CNNs

1. Train on Imagenet



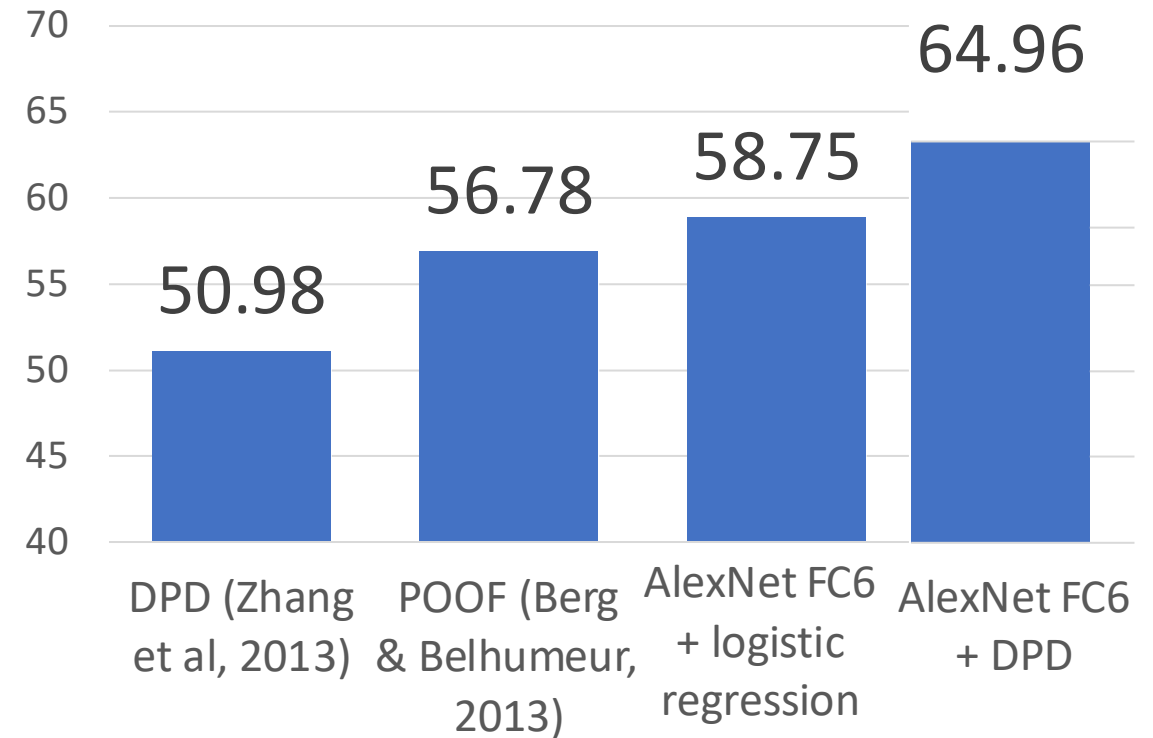
2. Use CNN as a feature extractor



Remove
last layer

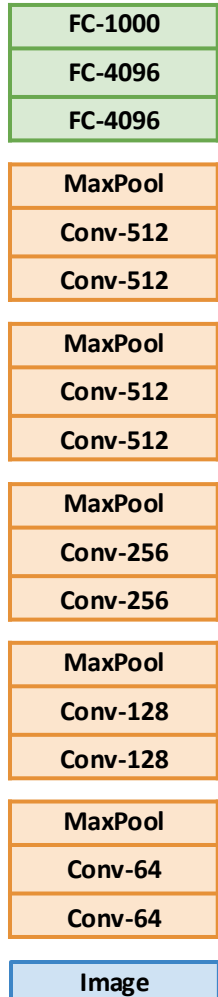
Freeze
these

Bird Classification on Caltech-UCSD



Transfer Learning with CNNs

1. Train on Imagenet



2. Use CNN as a feature extractor

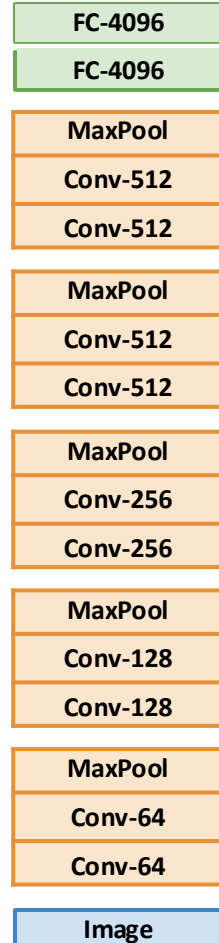
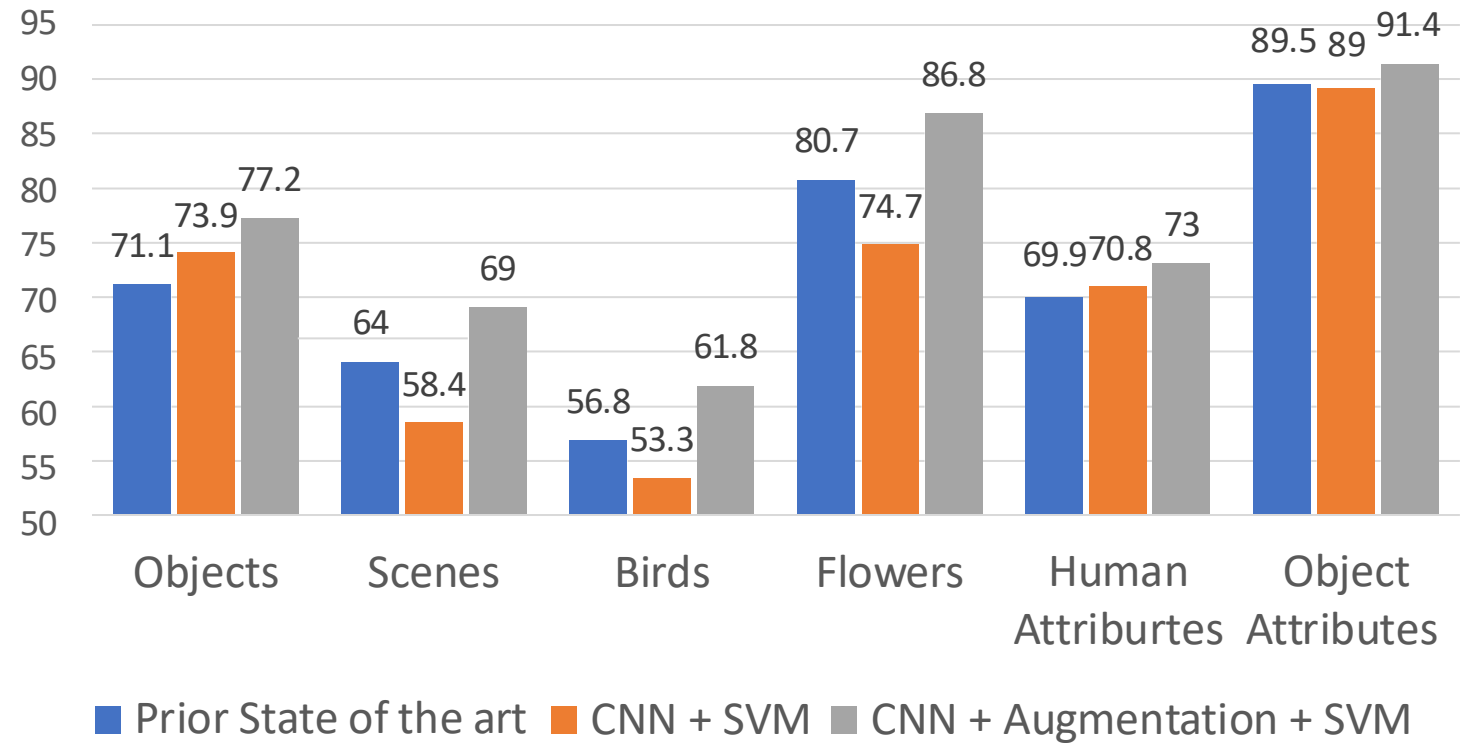
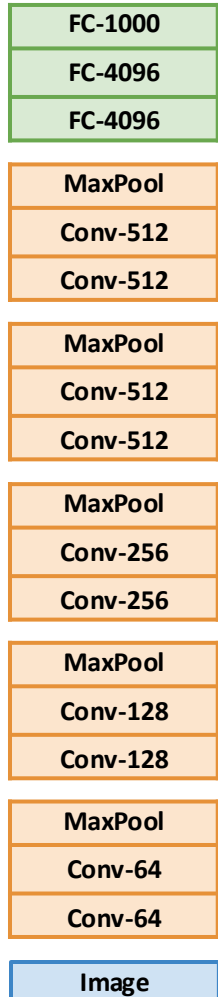


Image Classification



Transfer Learning with CNNs

1. Train on Imagenet



2. Use CNN as a feature extractor

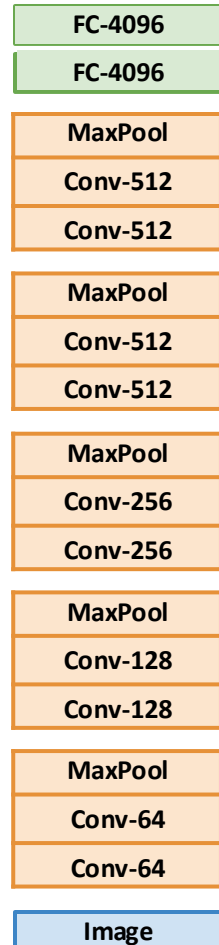
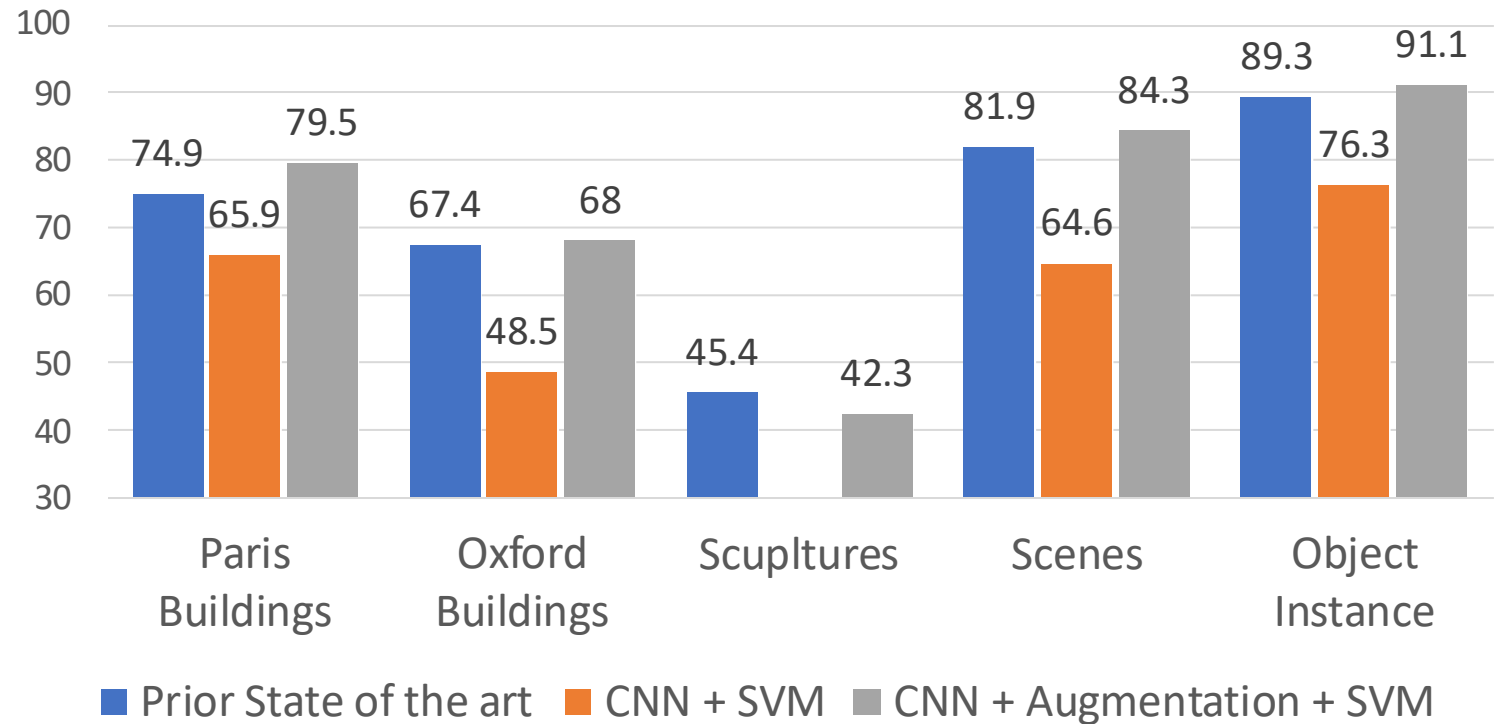
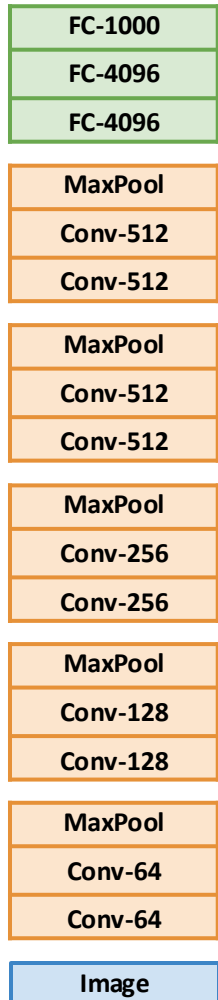


Image Retrieval: Nearest-Neighbor

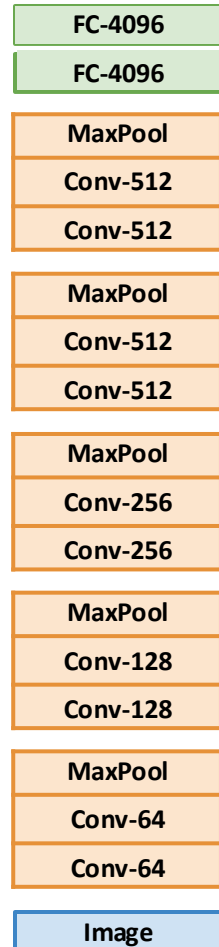


Transfer Learning with CNNs

1. Train on Imagenet



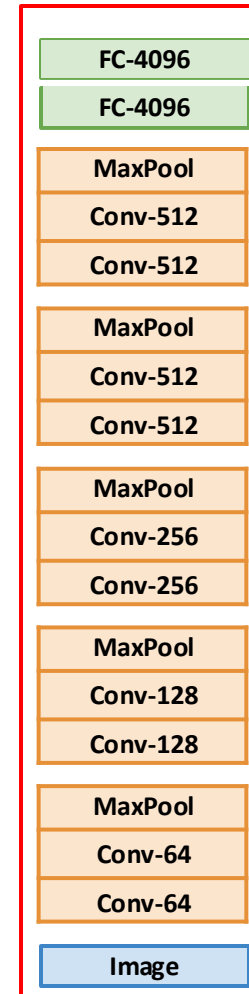
2. Use CNN as a feature extractor



Remove
last layer

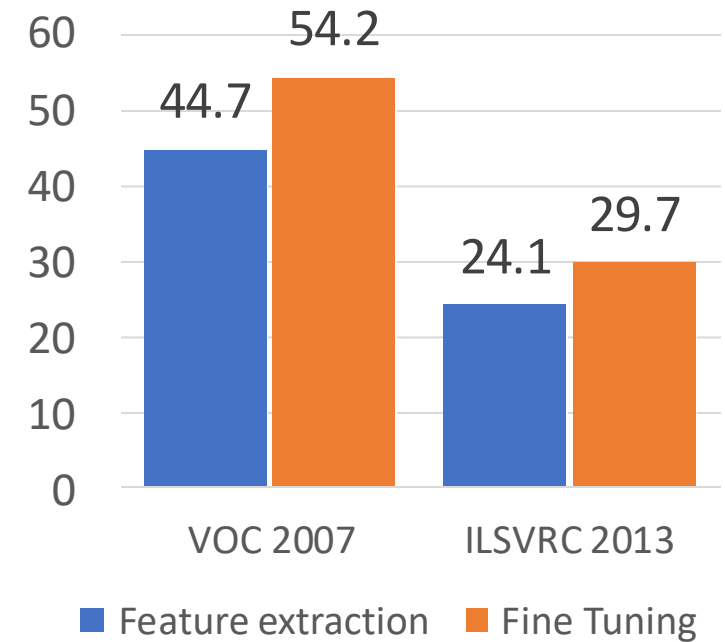
Freeze
these

3. Bigger dataset: Fine-Tuning



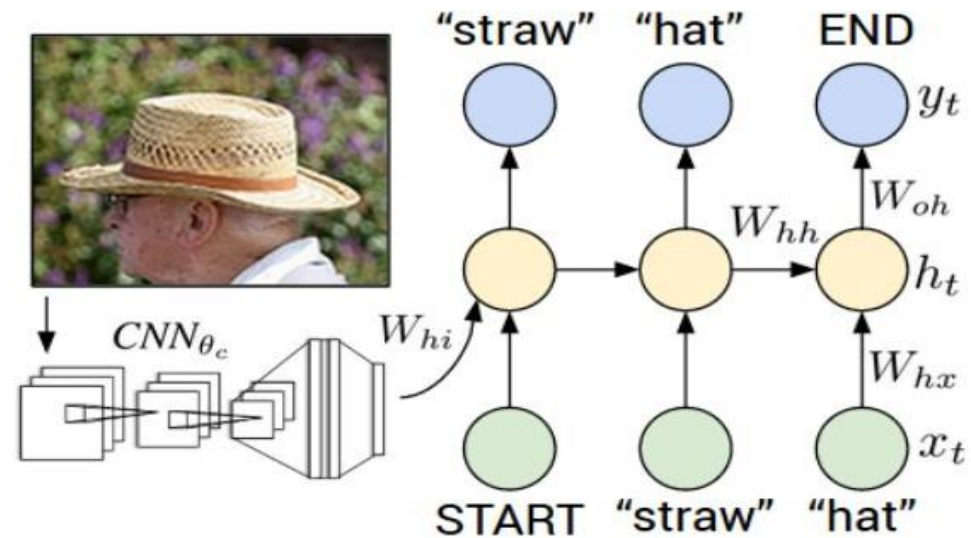
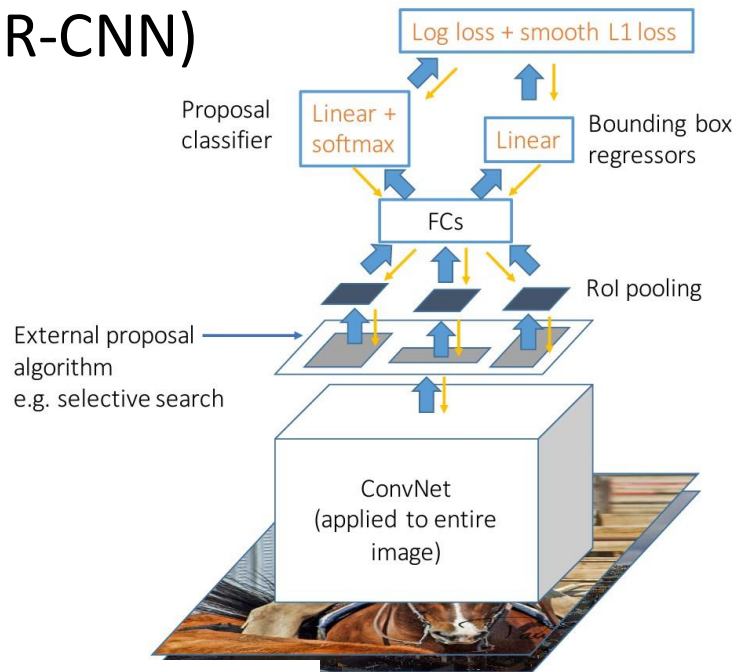
Continue training
CNN for new task!

Object Detection



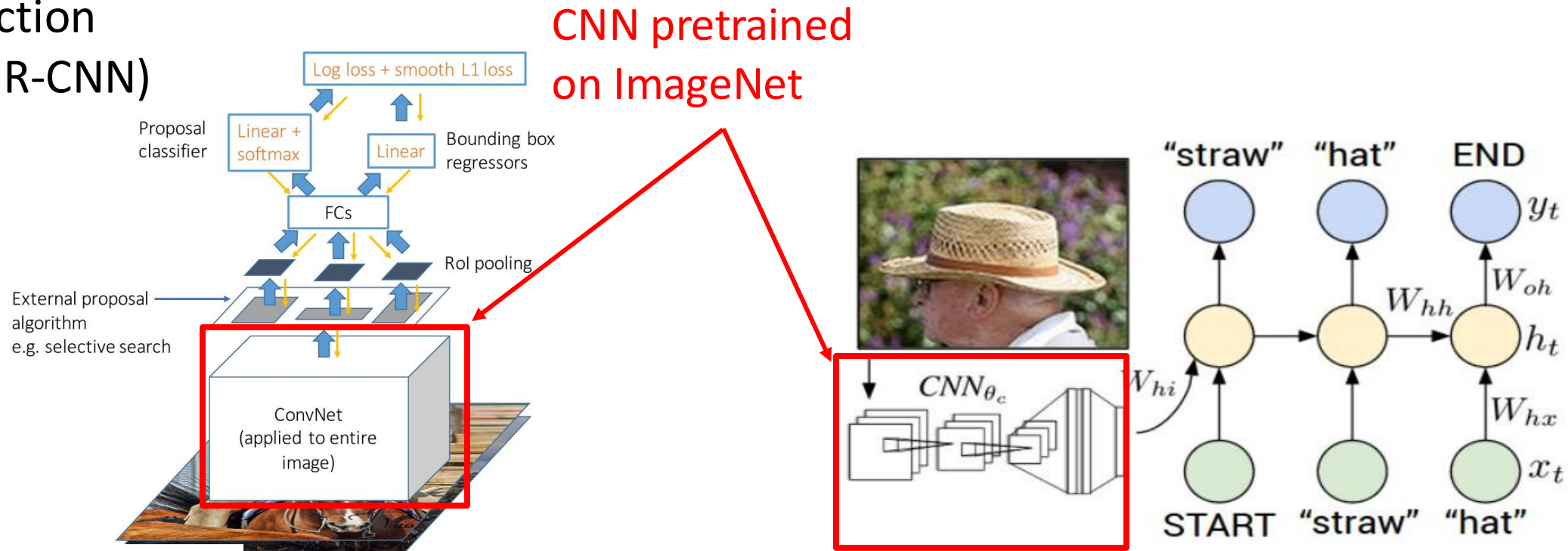
Transfer learning enabled solutions for more complex tasks; e.g., detection

Object Detection (Fast R-CNN)



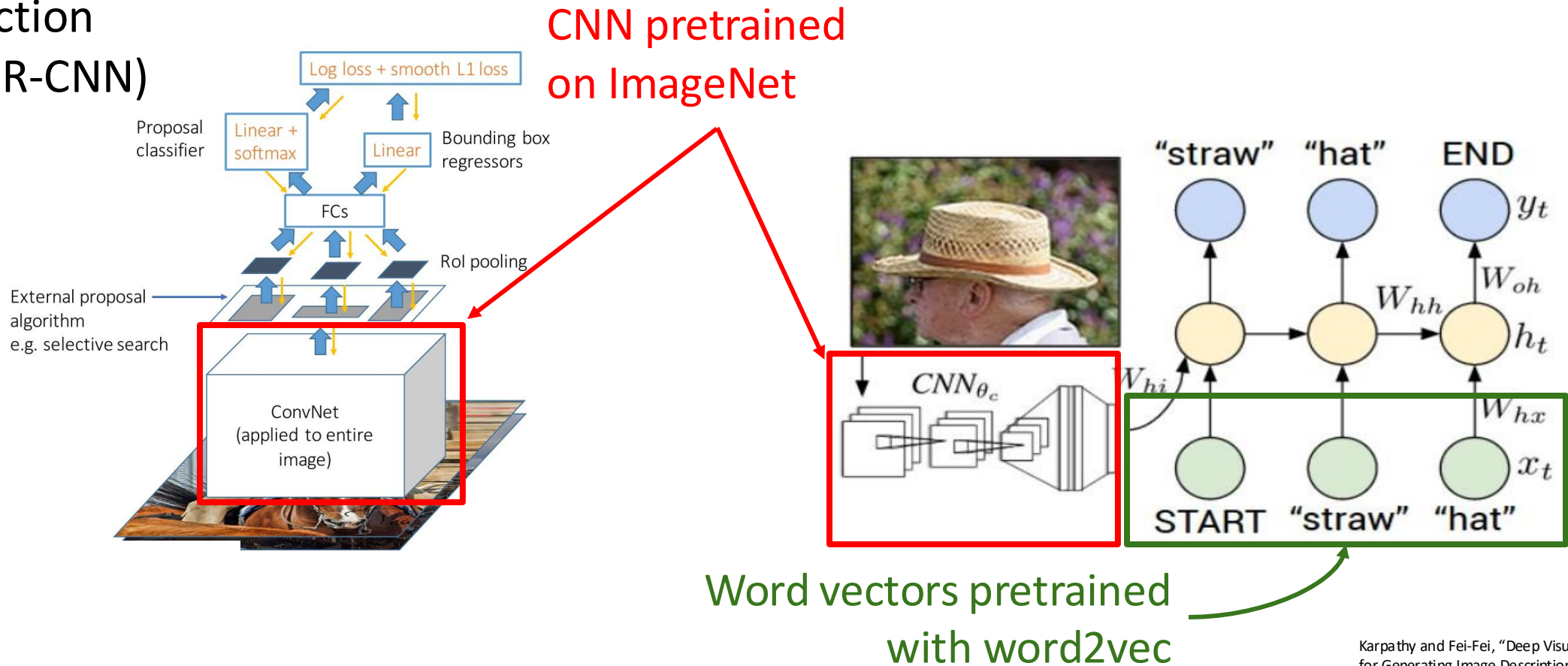
Transfer learning enabled solutions for more complex tasks; e.g., detection

Detection (Fast R-CNN)



Transfer learning enabled solutions for more complex tasks; e.g., detection

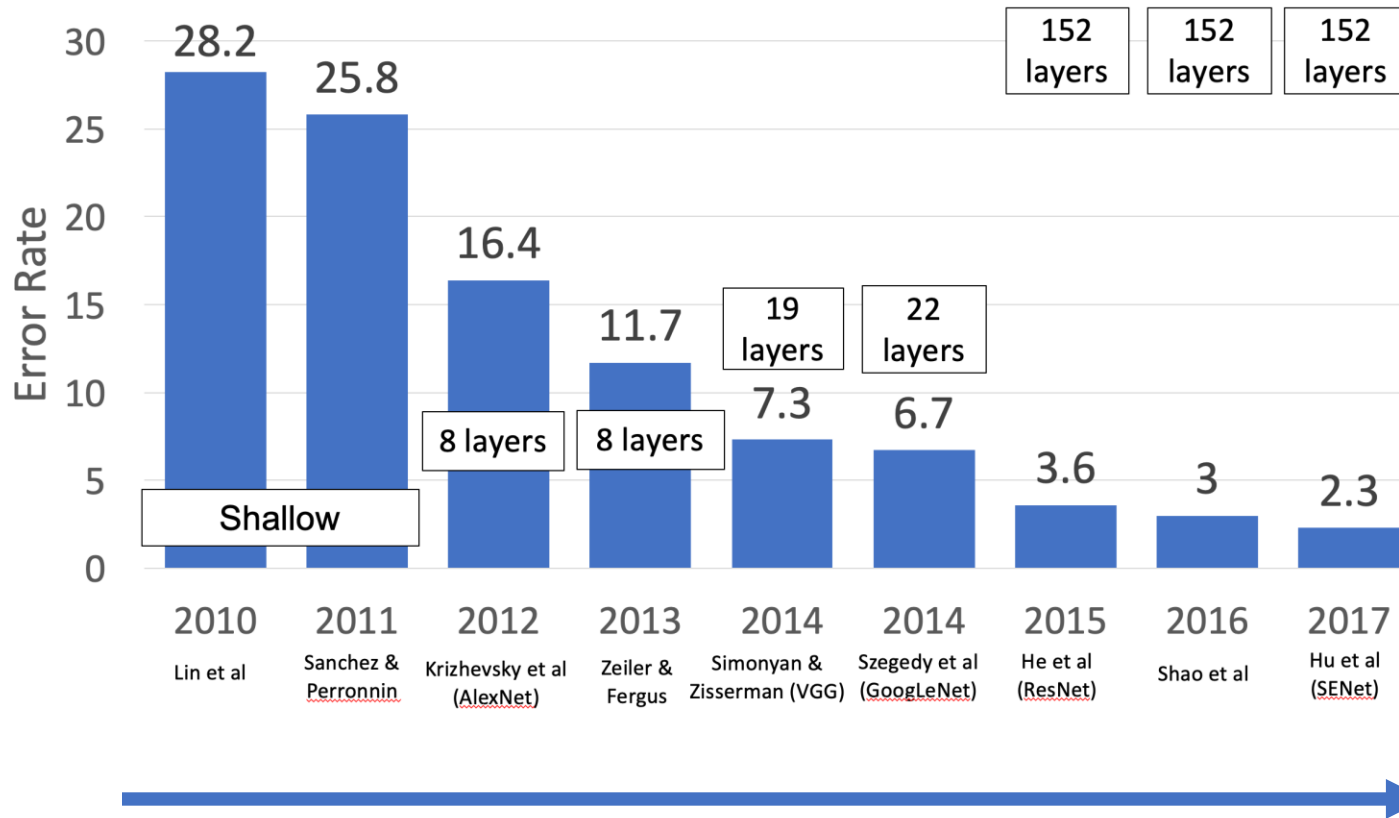
Detection (Fast R-CNN)



Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015

Transfer Learning with CNNs: Architecture Matters!

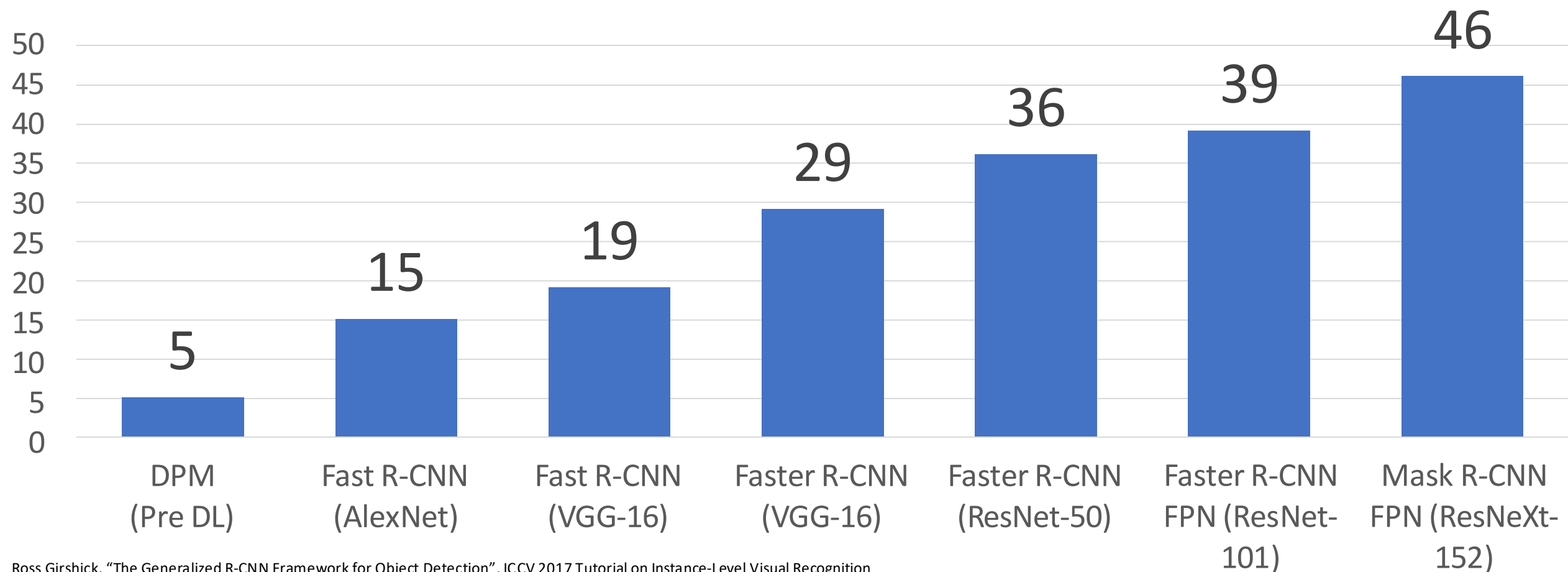
ImageNet Classification Challenge



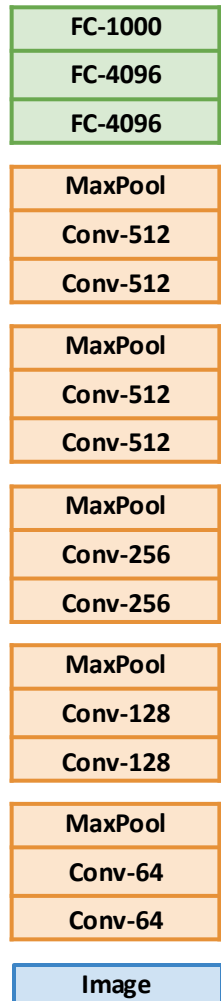
Improvements in CNN architectures lead to improvements in many downstream tasks thanks to transfer learning!

Transfer Learning with CNNs: Architecture Matters!

Object Detection on COCO

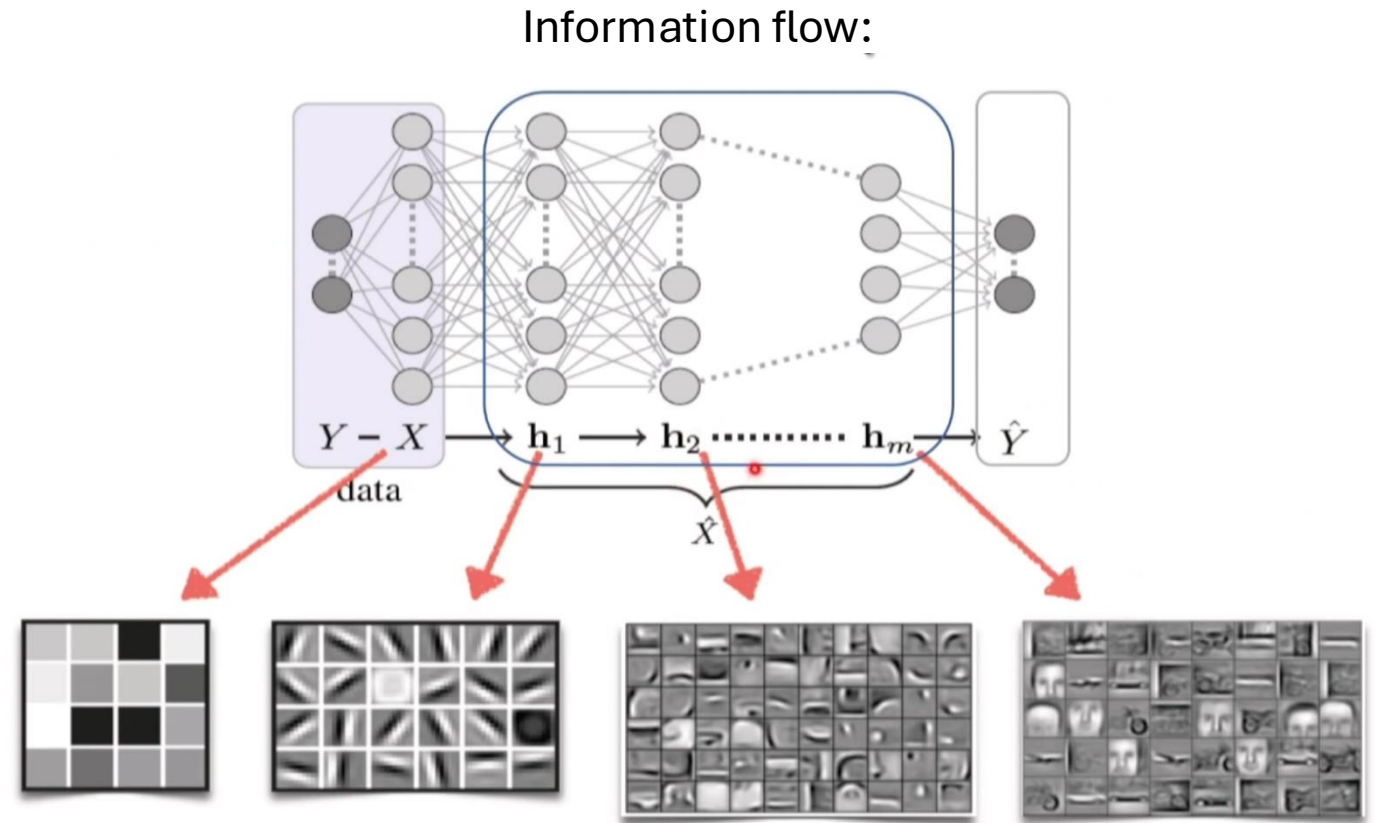


Transfer Learning with CNNs

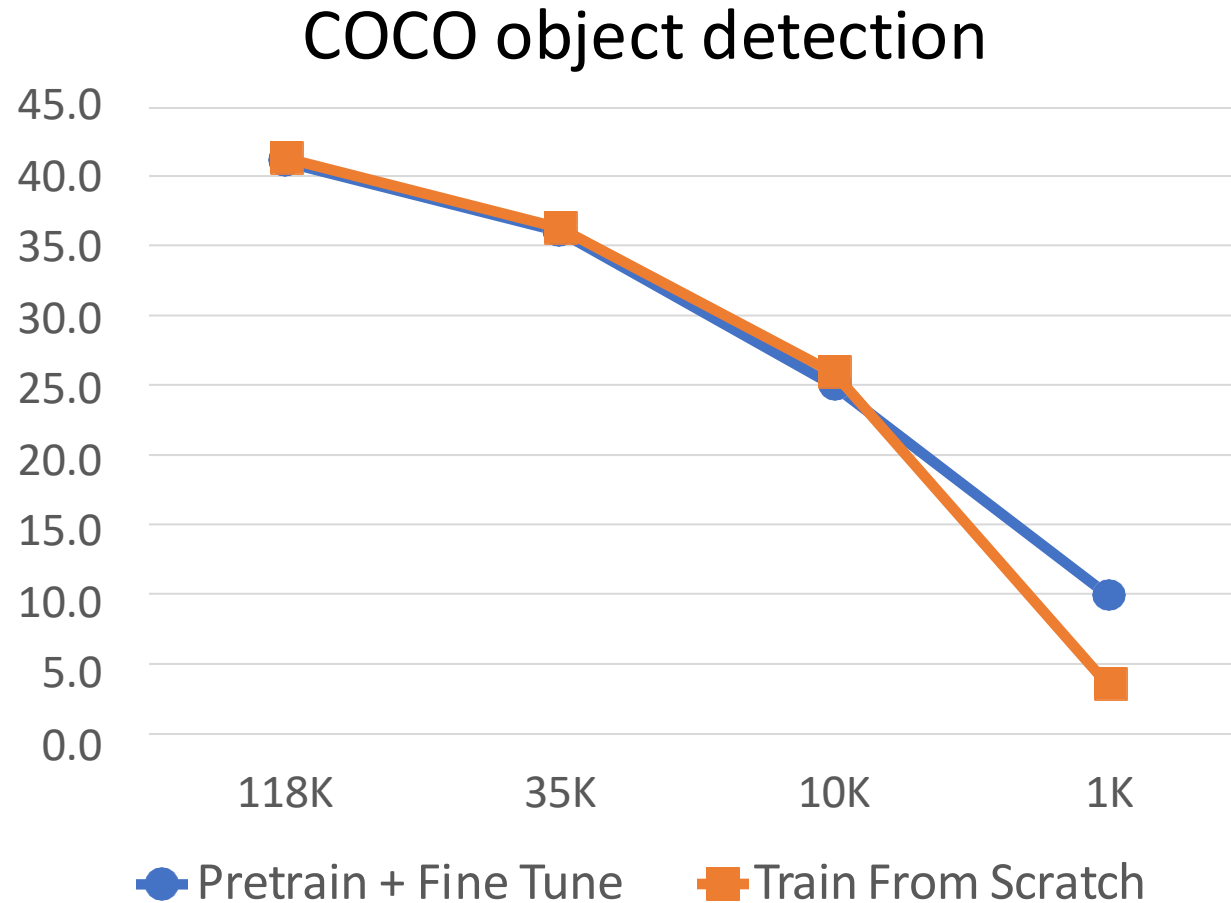


More specific
(better
describe Y)

More generic
(better
describe X)



Transfer learning is efficient; but not necessarily better



Pretraining + Finetuning beats training from scratch when dataset size is very small

Collecting more data is more effective than pre-training

Representation learning, challenges



Detected concepts:

Cow: 0.99, Pasture: 0.99, Grass: 0.99,
No Person: 0.98, Mammal: 0.98



Detected concepts:

No Person: 0.99, Water: 0.98, Beach:
0.97, Outdoors: 0.97, Seashore: 0.97

Representation learning, challenges

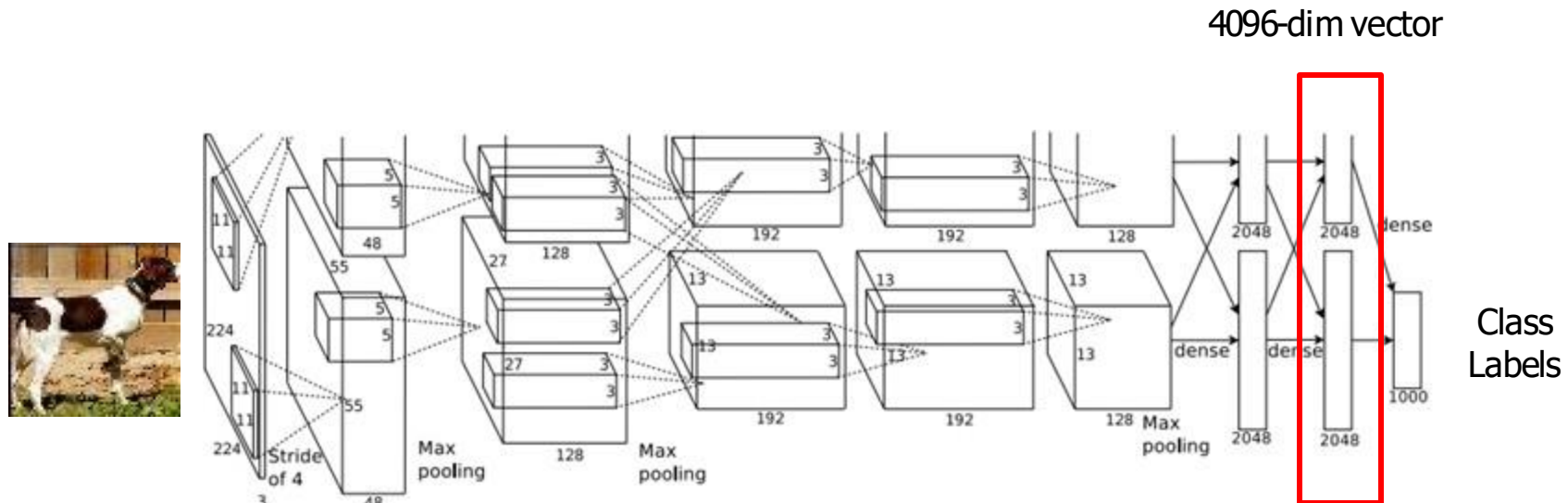
- ResNet-50 with ImageNet supervision

Labels Fraction	1%	10%	100%
Top1 Acc	25.4%	56.4%	78.9%
Top-5 Acc	48.4%	80.4%	94.2%

- 10% training data = 100,000 samples
- High workload for annotations

Learned representation

- What is the problem with large-scale training?
 - We need a lot of labeled data
- Is there a way we can train neural networks without the need for huge manually labeled datasets?



Self-supervised pretext tasks

- Example: learn to predict image transformations / complete corrupted images

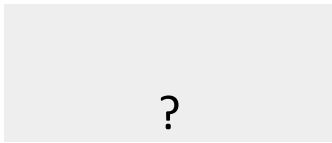
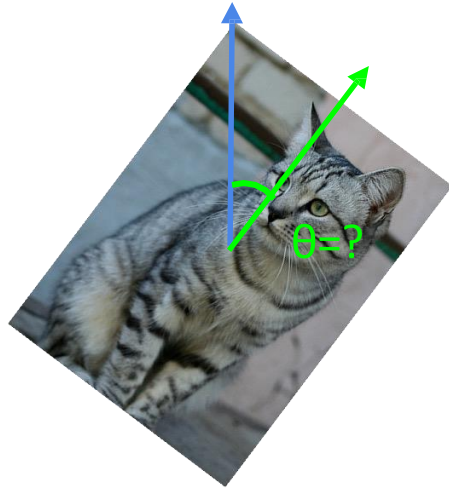


image completion



rotation prediction



“jigsaw puzzle”



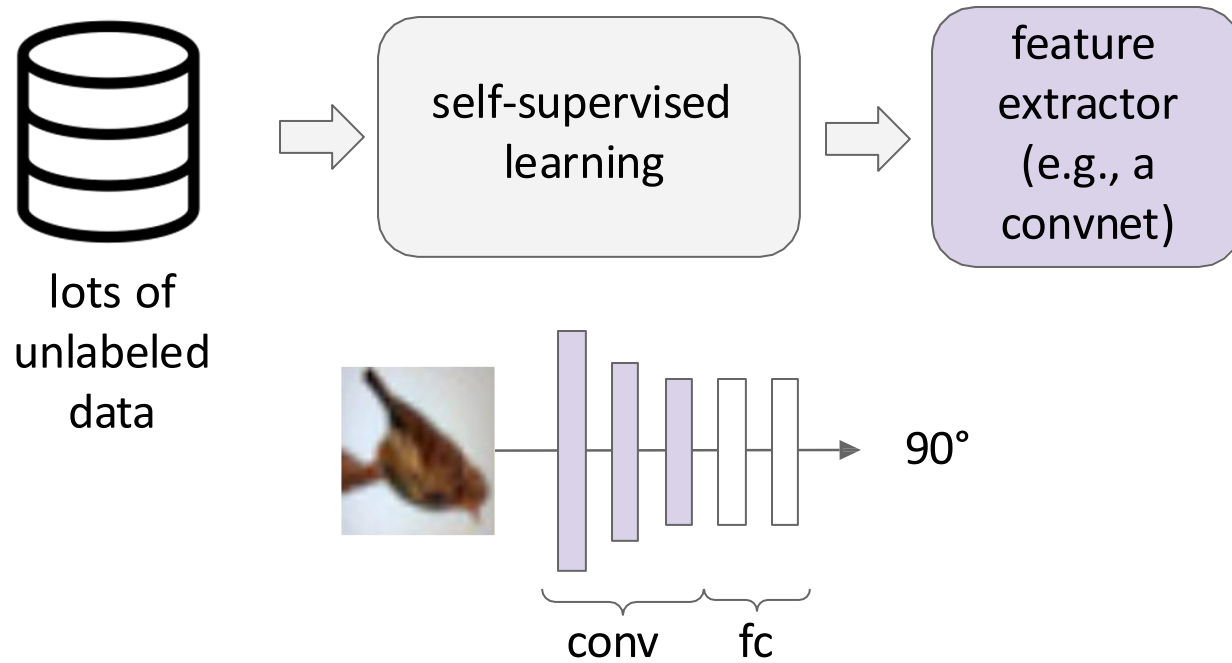
colorization

1. Solving the pretext tasks allow the model to learn good features.
2. We can automatically generate labels for the pretext tasks.

Benefits

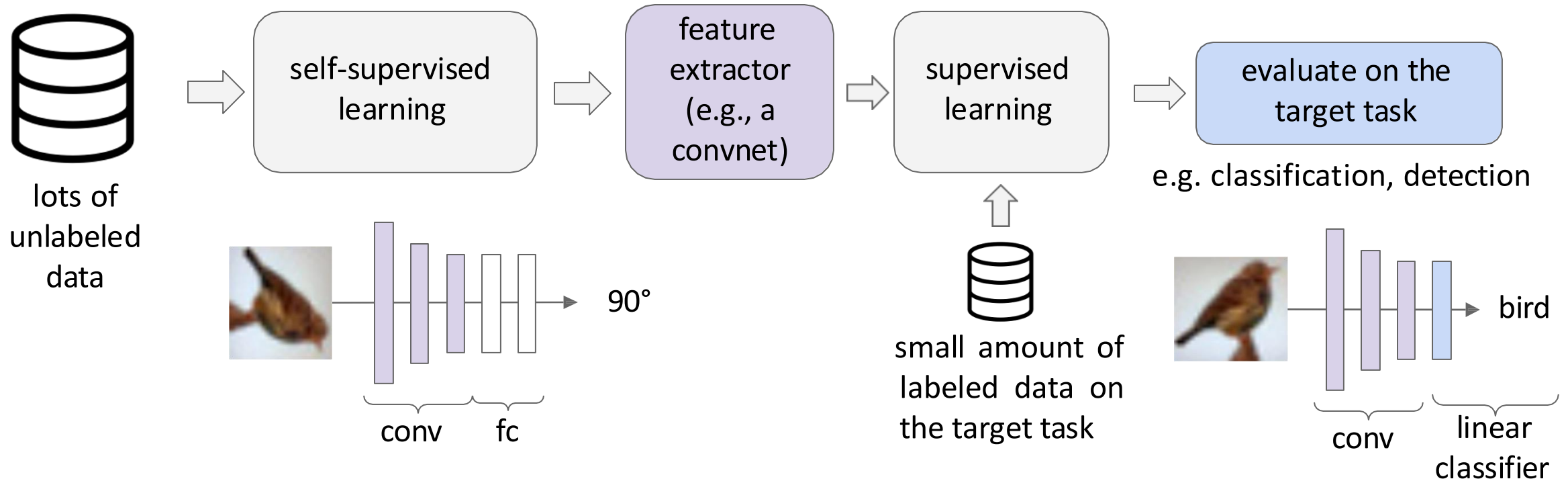
- Training data is “automatically generated”
- No need for human supervision
- More robust

How to evaluate a self-supervised learning method?



1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

How to evaluate a self-supervised learning method?



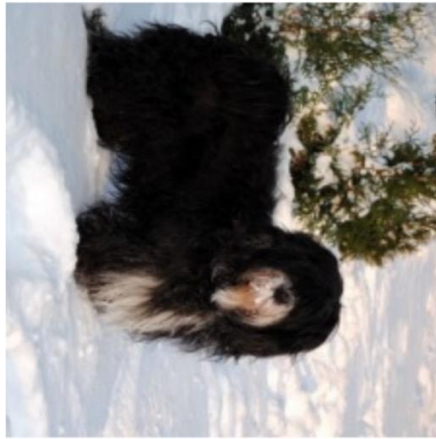
1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

2. Attach a shallow network on the feature extractor; train the shallow network on the target task with small amount of labeled data

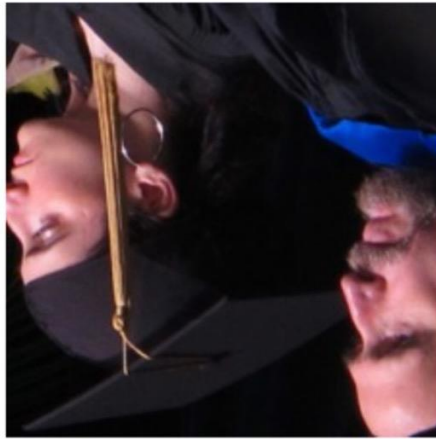
Pretext task: predict rotations



90° rotation



270° rotation



180° rotation



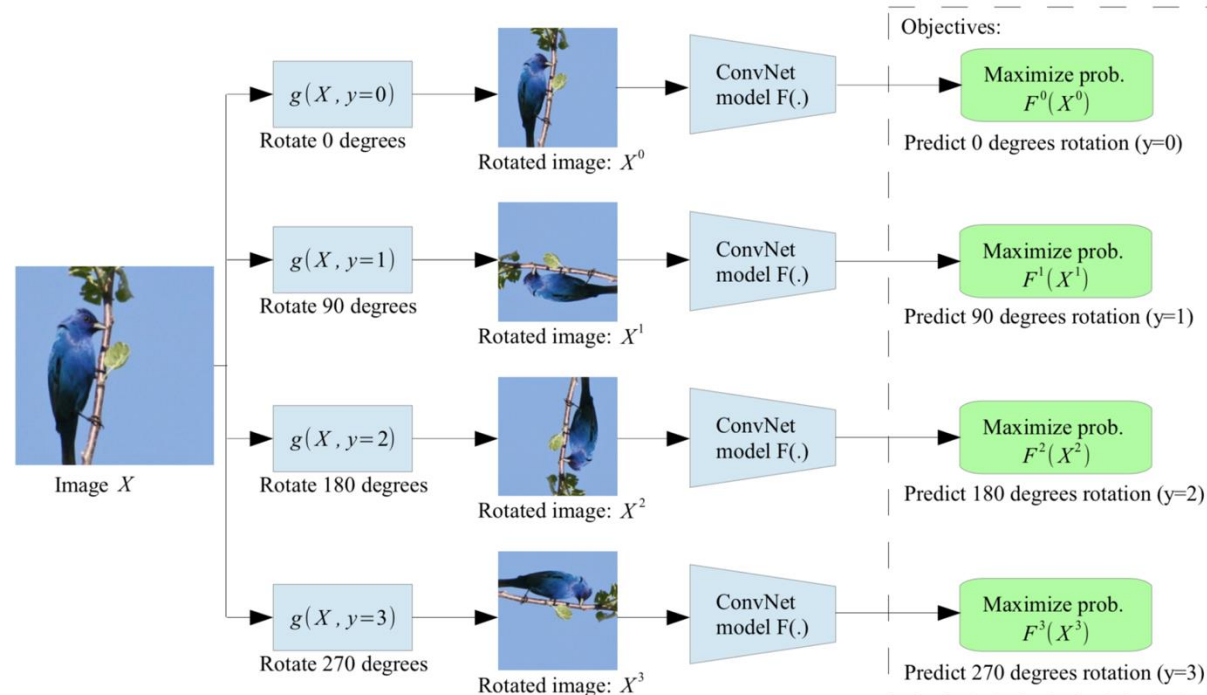
0° rotation



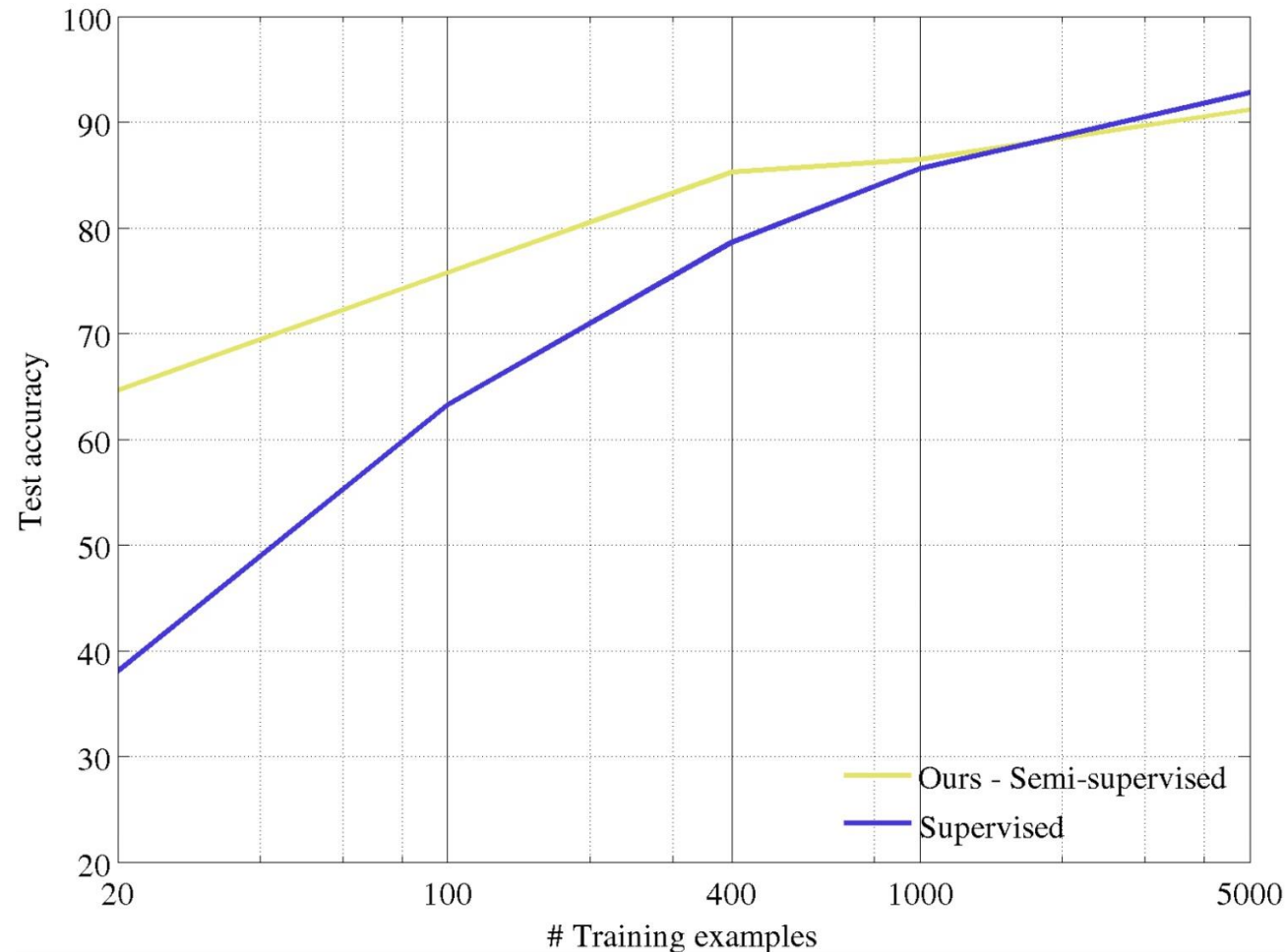
270° rotation

Rotations

- The model is trained to predict which rotation has been applied
- a 4-class classification problem



Evaluation on semi-supervised learning



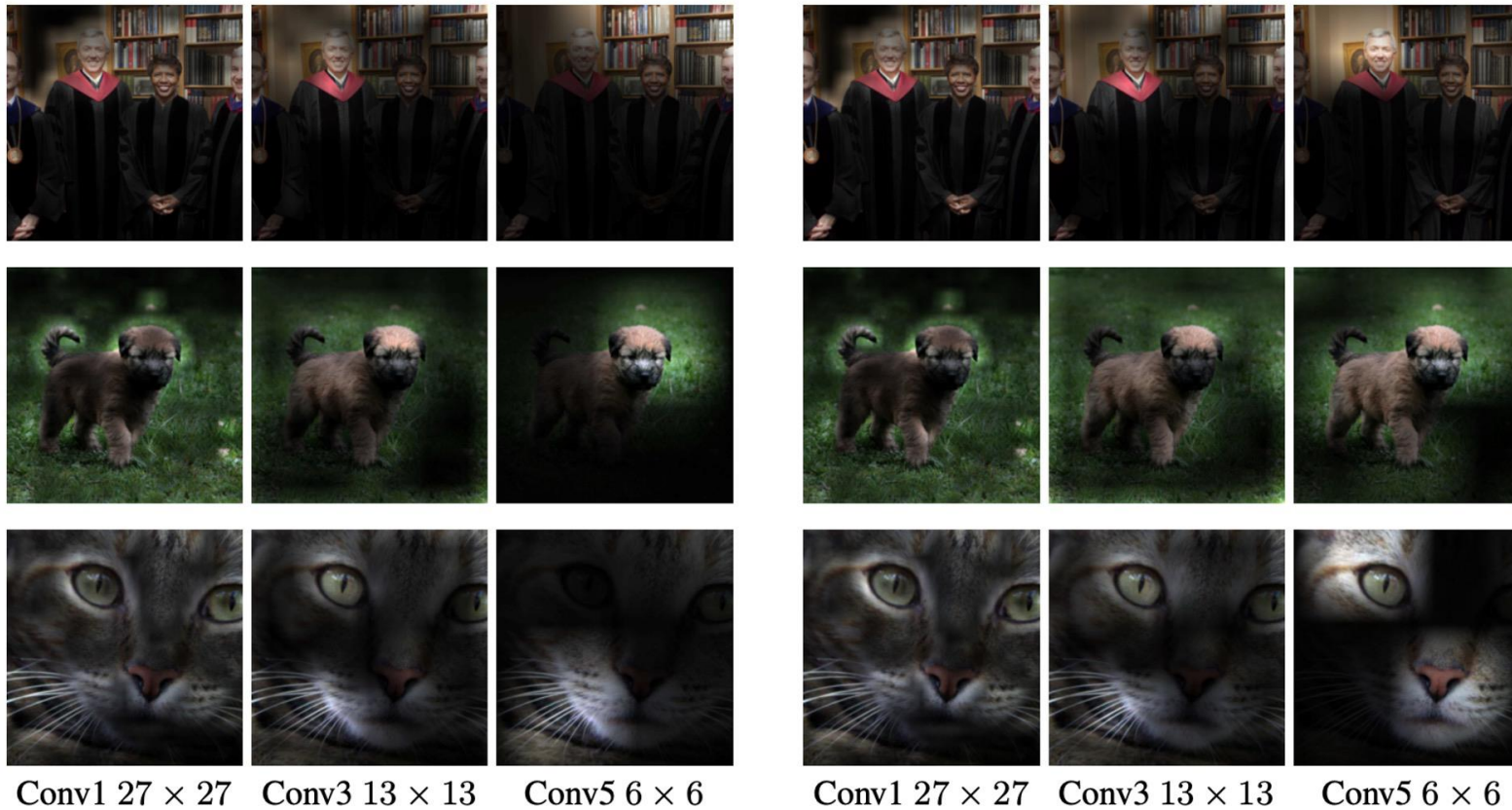
Self-supervised learning on CIFAR10 (entire training set).

Freeze conv1 + conv2

Learn conv3 + linear layers with subset of labeled CIFAR10 data (classification).

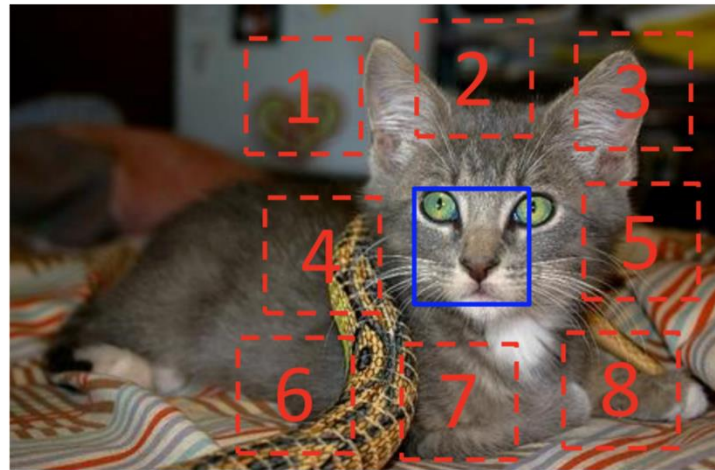
(Image source: [Gidaris et al. 2018](#))

What the model looked at (relevance maps)



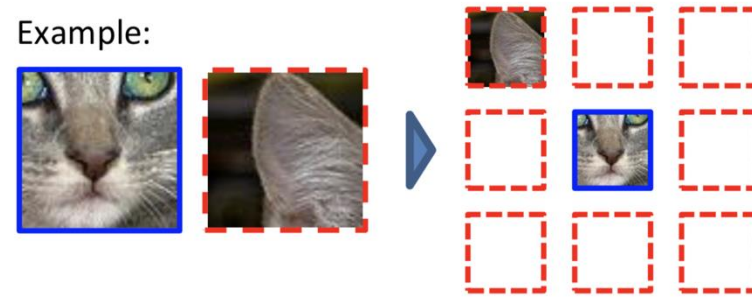
Patches

- Extract central patch and ask the model to predict the relationship between these patches.



$$X = \left(\begin{array}{c} \text{cat face} \\ \text{cat ear} \end{array} \right); Y = 3$$

Example:



Question 1:



Question 2:



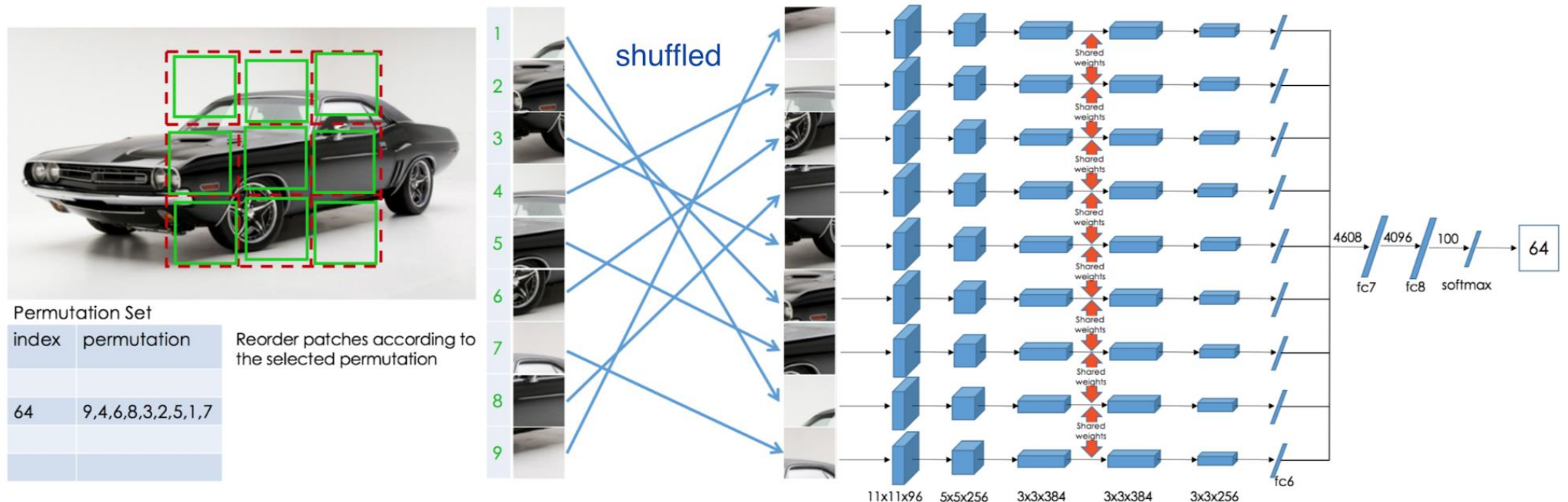
(Image source: [Doersch et al., 2015](#))

Self-Supervision – needs to be hard

- Add gaps between patches
- Randomly down sample some patches to as little as 100 total pixels, and then up sampling it
- Shift green and magenta toward gray or randomly drop 2 of 3 color channels

Jigsaw Puzzle

- Place 9 shuffled patches back to the original locations.



Transfer learned features to supervised learning

Table 1: Results on PASCAL VOC 2007 Detection and Classification. The results of the other methods are taken from Pathak *et al.* [30].

Method	Pretraining time	Supervision	Classification	Detection	Segmentation
Krizhevsky <i>et al.</i> [25]	3 days	1000 class labels	78.2%	56.8%	48.0%
Wang and Gupta[39]	1 week	motion	58.4%	44.0%	-
Doersch <i>et al.</i> [10]	4 weeks	context	55.3%	46.6%	-
Pathak <i>et al.</i> [30]	14 hours	context	56.5%	44.5%	29.7%
Ours	2.5 days	context	67.6%	53.2%	37.6%

“Ours” is feature learned from solving image Jigsaw puzzles (Noroozi & Favaro, 2016). Doersch et al. is the method with relative patch location

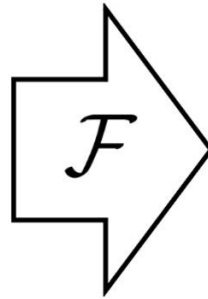
(source: [Noroozi & Favaro, 2016](#))

Image coloring



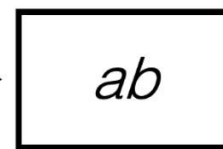
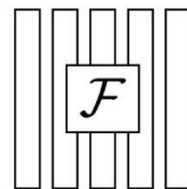
Grayscale image: L channel

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 1}$$

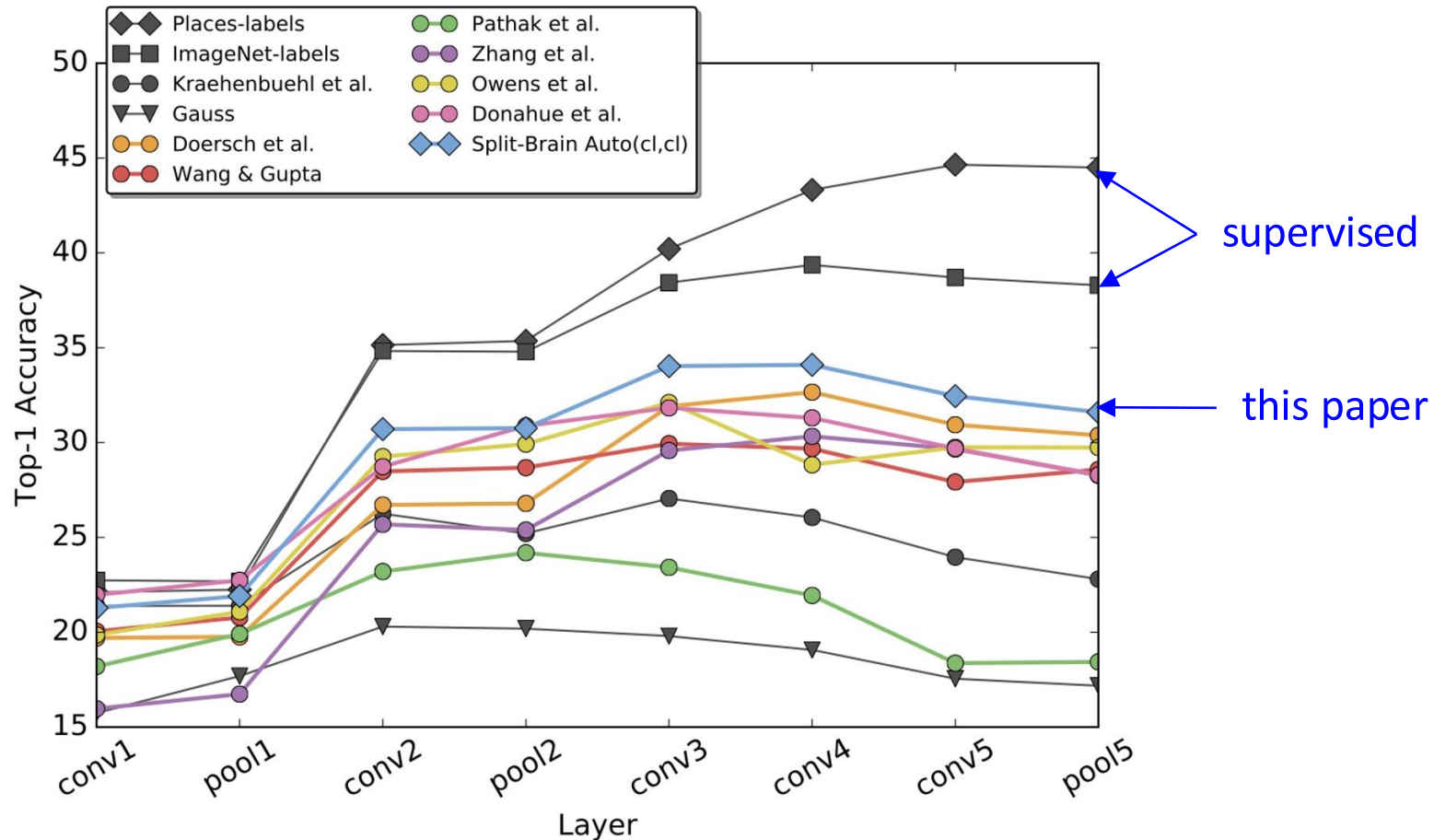


Concatenate (L, ab) channels

$$(\mathbf{X}, \hat{\mathbf{Y}})$$



Transfer learned features to supervised learning



Self-supervised learning on ImageNet (entire training set).

Use concatenated features from F_1 and F_2

Labeled data is from the Places (Zhou 2016).

Combine Pretext tasks

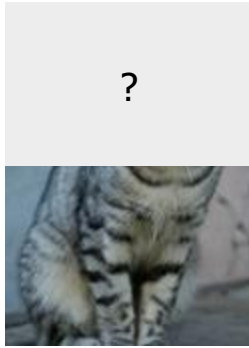
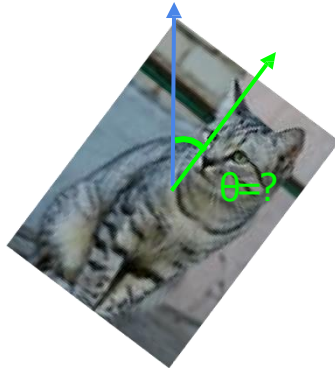
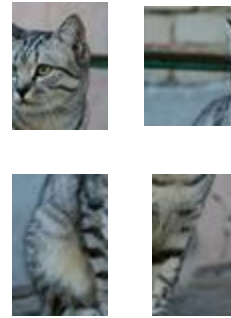


image completion



rotation prediction



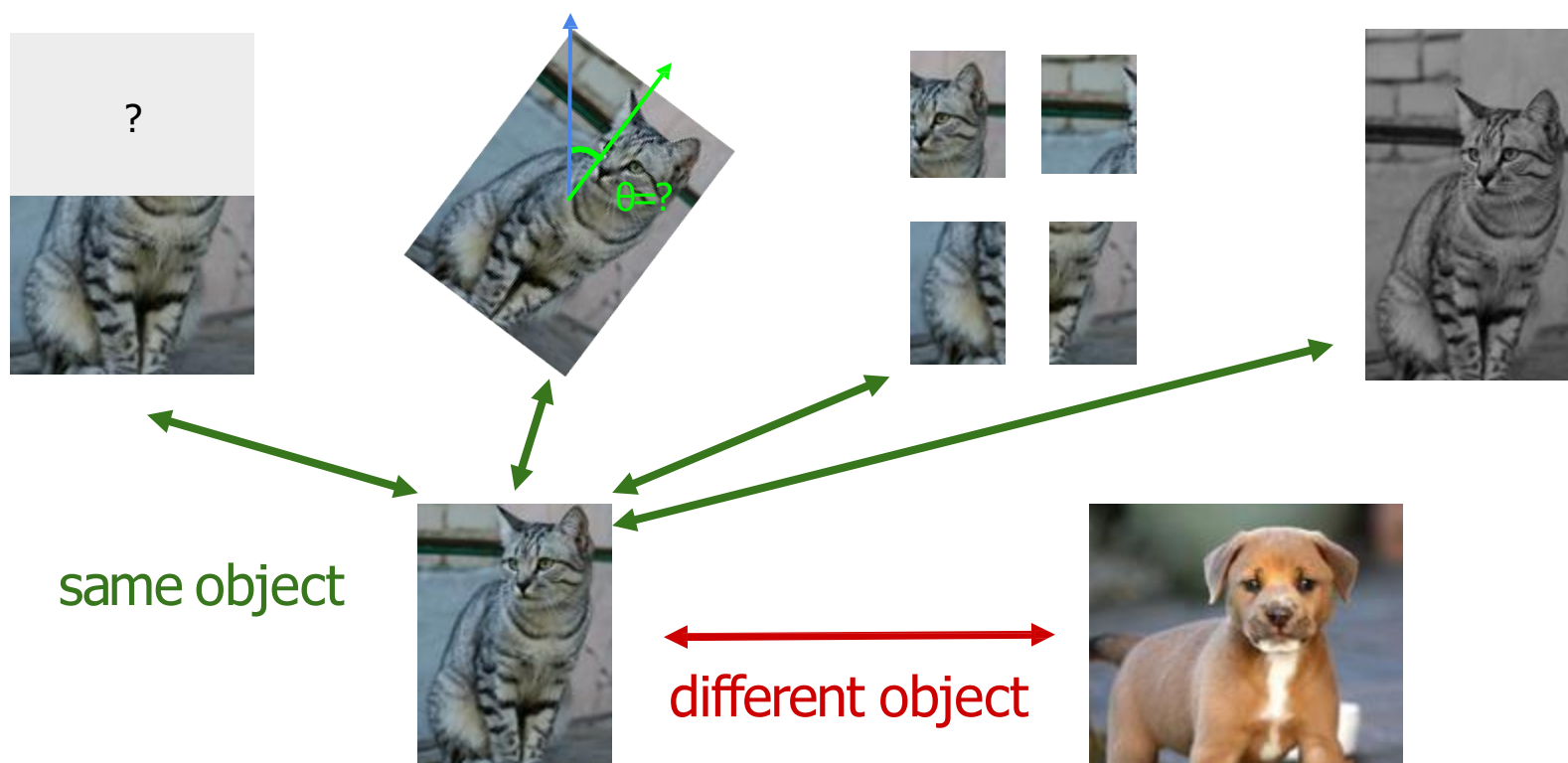
"jigsaw puzzle"



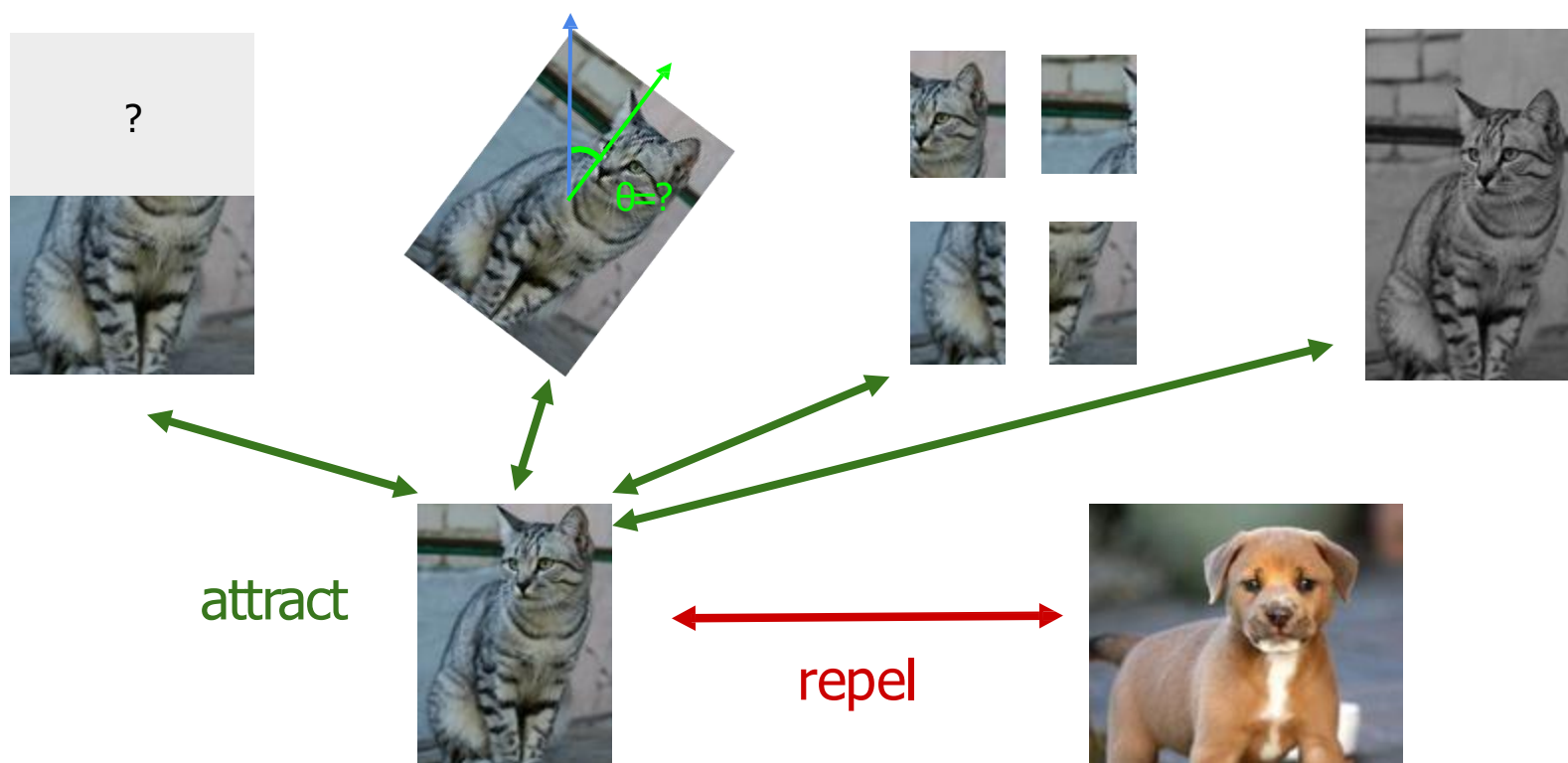
colorization

Issue: Learned representations may be tied to a specific pretext task!

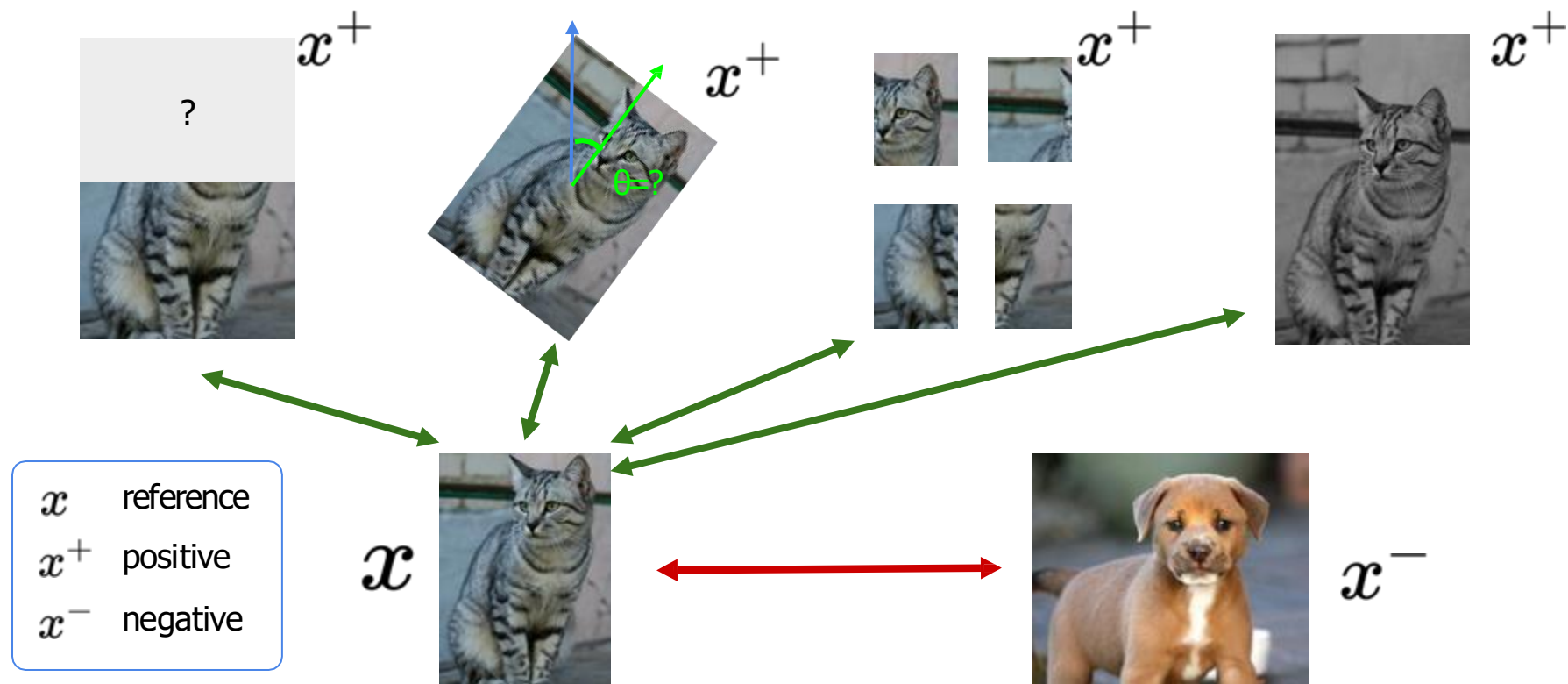
A more general pretext, negatives



A more general pretext, negatives



Contrastive Representation Learning



What we want

- Given a chosen score function, we aim to learn an encoder function f that yields high score for positive pairs (x, x^+) and low scores for negative pairs (x, x^-) :

$$\text{score}(f(x), f(x^+)) > \text{score}(f(x), f(x^-))$$

- x reference sample
- x^+ positive sample
- x^- negative sample

Contrastive Loss

- Loss given a positive sample and $N - 1$ negative samples:

$$\mathcal{L} = -\mathbb{E} \left[\log \frac{\exp \left(s(f(x), f(x^+)) \right)}{\exp \left(s(f(x), f(x^+)) \right) + \sum_{j \in N-1} \exp \left(s(f(x), f(x_j^-)) \right)} \right]$$



x



x^+



x



x_1^-



x_2^-



x_3^-

...

Contrastive Loss

- Loss given a positive sample and $N - 1$ negative samples:

$$\mathcal{L} = -\mathbb{E} \left[\log \frac{\overset{\text{score for the positive pair}}{\exp \left(s(f(x), f(x^+)) \right)}}{\exp \left(s(f(x), f(x^+)) \right) + \sum_{j \in N-1} \exp \left(s(f(x), f(x^-)) \right)} \right]$$

score for the N-1 negative pairs

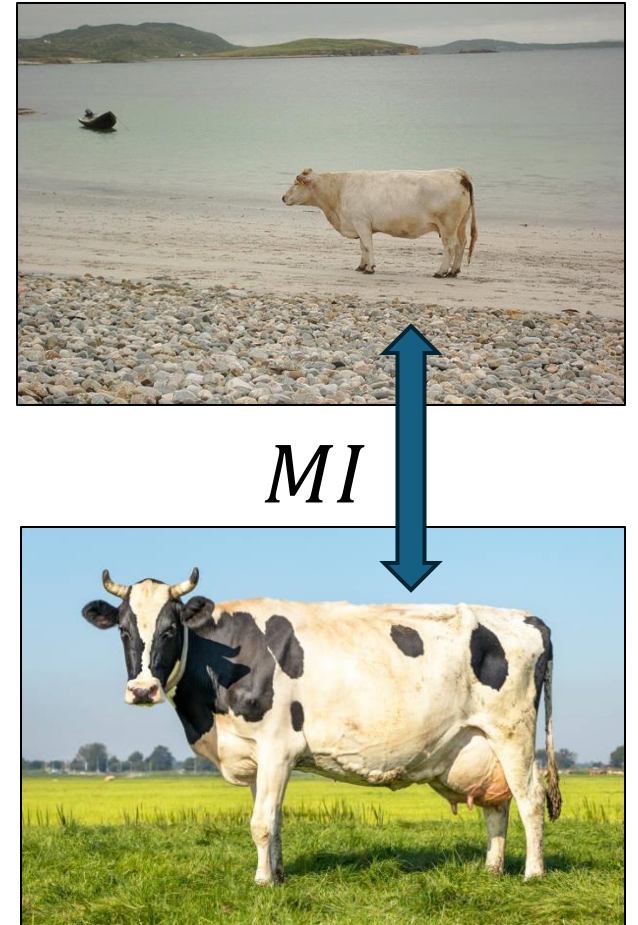
- Intuetivly, maximize the positive score from the N samples

Contrastive Loss relates to Mutual Information

- Formally known as the InfoNCE loss
- A lower bound on the mutual information between $f(x)$, $f(x^+)$

$$MI[f(x), f(x^+)] \geq -L + \log(N)$$

- The larger the negative sample size (N), the tighter the bound



SimCLR: A framework for data augmentations



(a) Original



(b) Crop and resize



(c) Crop, resize (and flip)



(d) Color distort. (drop)



(e) Color distort. (jitter)



(f) Rotate $\{90^\circ, 180^\circ, 270^\circ\}$



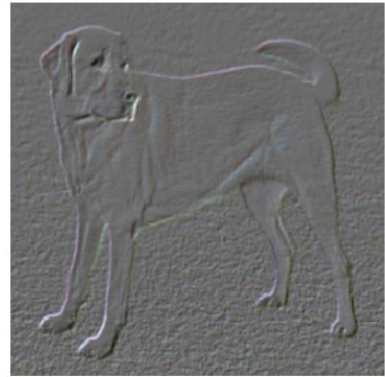
(g) Cutout



(h) Gaussian noise



(i) Gaussian blur



(j) Sobel filtering

SimCLR: A Simple Framework for Contrastive Learning

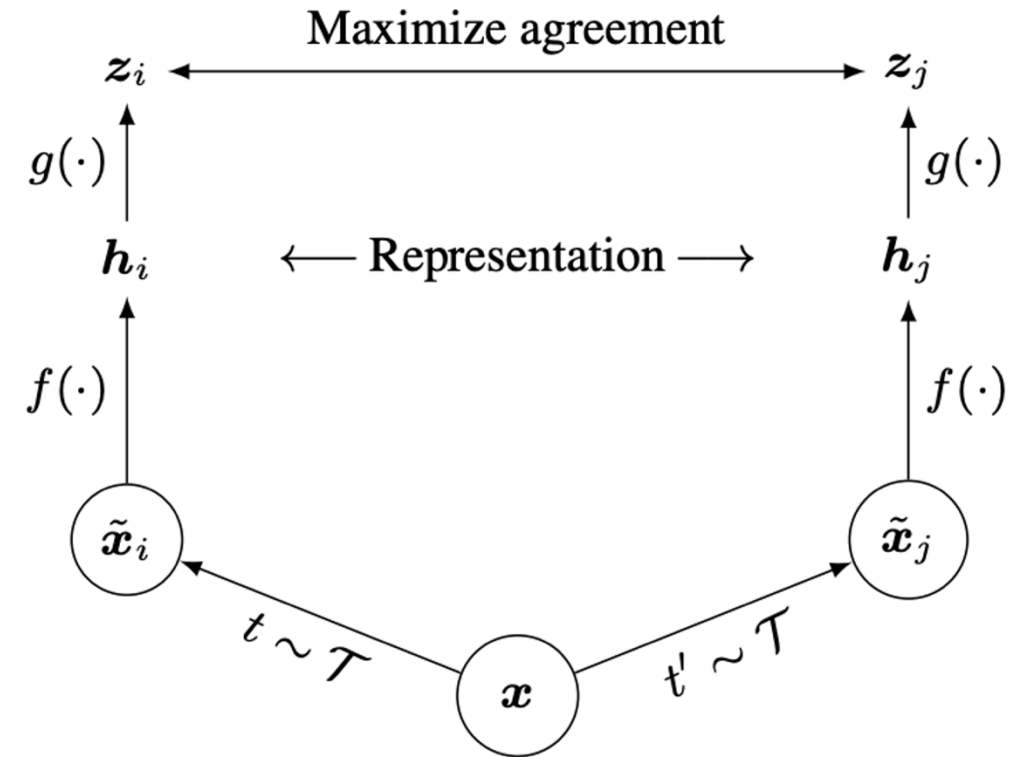
Cosine similarity as the score function:

$$s(u, v) = \frac{u^T v}{||u|| ||v||}$$

Use a projection network $g(\cdot)$ to project features to a space where contrastive learning is applied

Generate positive samples through data augmentation:

- random cropping, random color distortion, and random blur.



Animation

Code

Generate a positive pair
by sampling data
augmentation functions

Algorithm 1 SimCLR's main learning algorithm.

```
input: batch size  $N$ , constant  $\tau$ , structure of  $f, g, \mathcal{T}$ .  
for sampled minibatch  $\{\mathbf{x}_k\}_{k=1}^N$  do  
  for all  $k \in \{1, \dots, N\}$  do  
    draw two augmentation functions  $t \sim \mathcal{T}, t' \sim \mathcal{T}$   
    # the first augmentation  
     $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$   
     $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$  # representation  
     $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$  # projection  
    # the second augmentation  
     $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$   
     $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$  # representation  
     $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$  # projection  
  end for  
  for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do  
     $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$  # pairwise similarity  
  end for  
  define  $\ell(i, j)$  as  $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$   
   $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$   
  update networks  $f$  and  $g$  to minimize  $\mathcal{L}$   
end for  
return encoder network  $f(\cdot)$ , and throw away  $g(\cdot)$ 
```

Code

Algorithm 1 SimCLR's main learning algorithm.

```
input: batch size  $N$ , constant  $\tau$ , structure of  $f, g, \mathcal{T}$ .  
for sampled minibatch  $\{\mathbf{x}_k\}_{k=1}^N$  do  
  for all  $k \in \{1, \dots, N\}$  do  
    draw two augmentation functions  $t \sim \mathcal{T}, t' \sim \mathcal{T}$   
    # the first augmentation  
     $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$   
     $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$  # representation  
     $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$  # projection  
    # the second augmentation  
     $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$   
     $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$  # representation  
     $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$  # projection  
  end for  
  for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do  
     $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$  # pairwise similarity  
  end for  
  define  $\ell(i, j)$  as  $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$   
   $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$   
  update networks  $f$  and  $g$  to minimize  $\mathcal{L}$   
end for  
return encoder network  $f(\cdot)$ , and throw away  $g(\cdot)$ 
```

Generate a positive pair
by sampling data
augmentation functions

InfoNCE loss:
Use all non-
positive samples
in the batch
as $\mathbf{x}^-$

Code

Algorithm 1 SimCLR's main learning algorithm.

```
input: batch size  $N$ , constant  $\tau$ , structure of  $f, g, \mathcal{T}$ .  
for sampled minibatch  $\{\mathbf{x}_k\}_{k=1}^N$  do  
  for all  $k \in \{1, \dots, N\}$  do  
    draw two augmentation functions  $t \sim \mathcal{T}, t' \sim \mathcal{T}$   
    # the first augmentation  
     $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$   
     $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$  # representation  
     $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$  # projection  
    # the second augmentation  
     $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$   
     $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$  # representation  
     $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$  # projection  
  end for  
  for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do  
     $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$  # pairwise similarity  
  end for  
  define  $\ell(i, j)$  as  $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$   
   $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$   
  update networks  $f$  and  $g$  to minimize  $\mathcal{L}$   
end for  
return encoder network  $f(\cdot)$ , and throw away  $g(\cdot)$ 
```

Generate a positive pair
by sampling data
augmentation functions

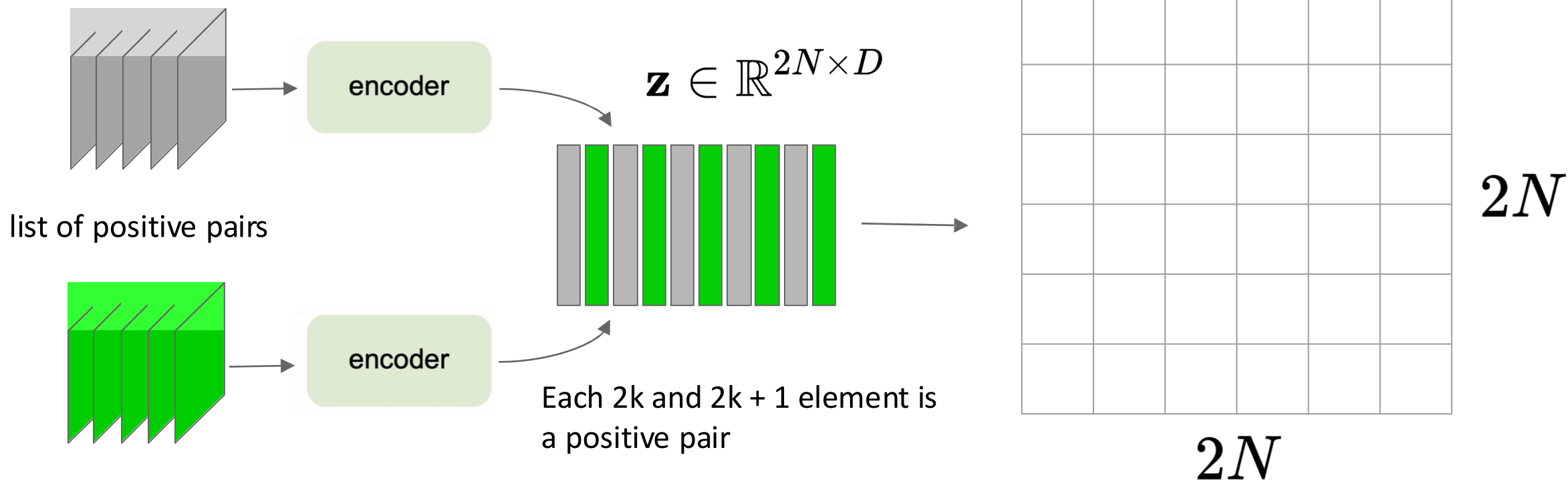
Iterate through and
use each of the $2N$
sample as reference,
compute average loss

InfoNCE loss:
Use all non-
positive samples
in the batch
as $\mathbf{x}^-$

SimCLR: mini-batch training

$$s_{i,j} = \frac{z_i^T z_j}{||z_i|| ||z_j||}$$

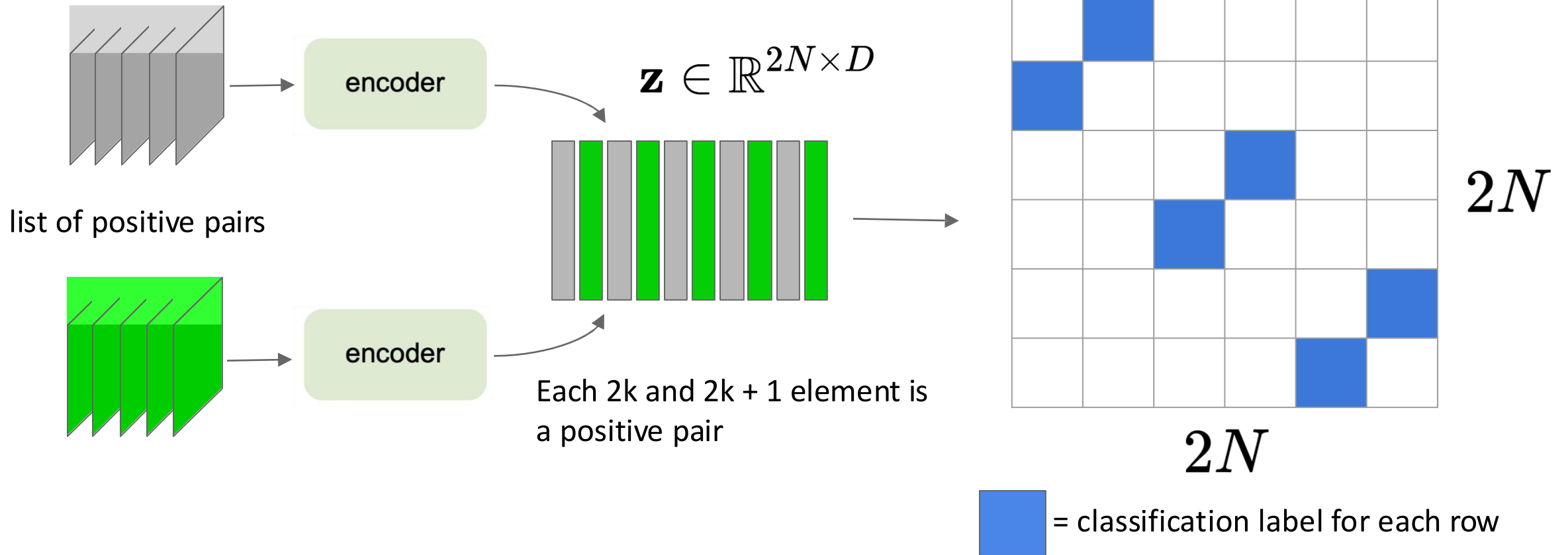
“Affinity matrix”



SimCLR: mini-batch training

$$s_{i,j} = \frac{z_i^T z_j}{||z_i|| ||z_j||}$$

"Affinity matrix"



SimCLR results

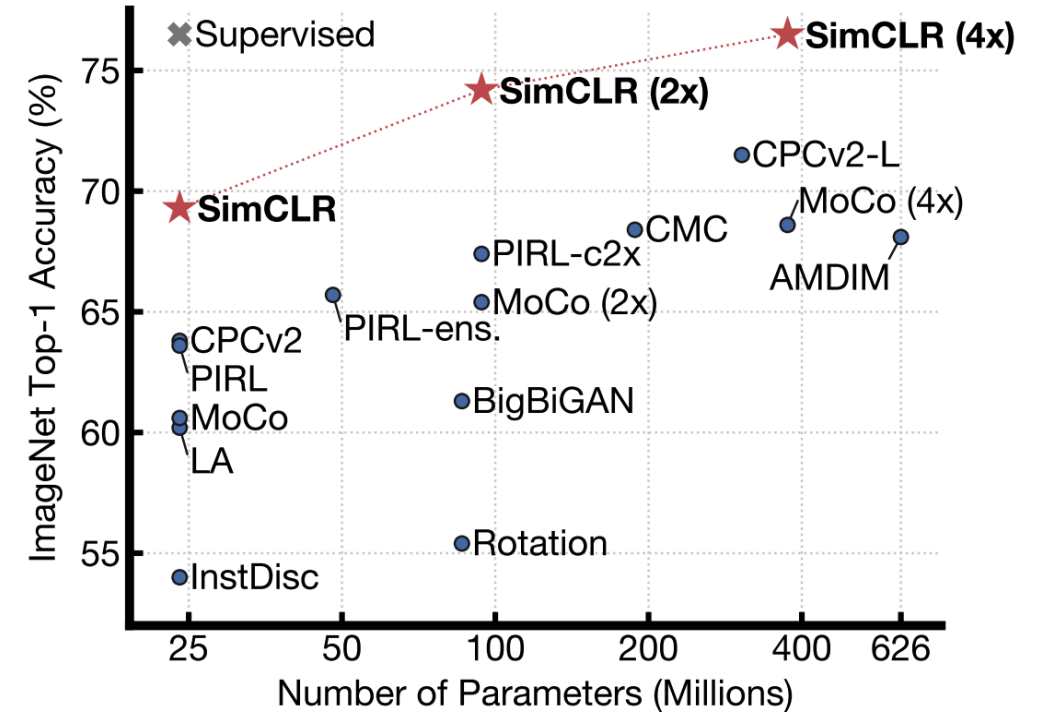
- Supervised:

Labels Fraction	1%	10%	100%
Top1 Acc	25.4	56.4	78.4
Top-5 Acc	48.4	80.4	94.2

- Self-Supervised + Linear Classifier w\ ImageNet:

Labels Fraction	1%	10%	100%
Top1 Acc	48.3	65.6	69.3
Top-5 Acc	75.5	87.8	89.0

Self-Supervised features are as useful as supervised features:



Large batch size

- Large training batch size is crucial
- Increases mutual information

$$MI[f(x), f(x^+)] \geq -L + \log(N)$$

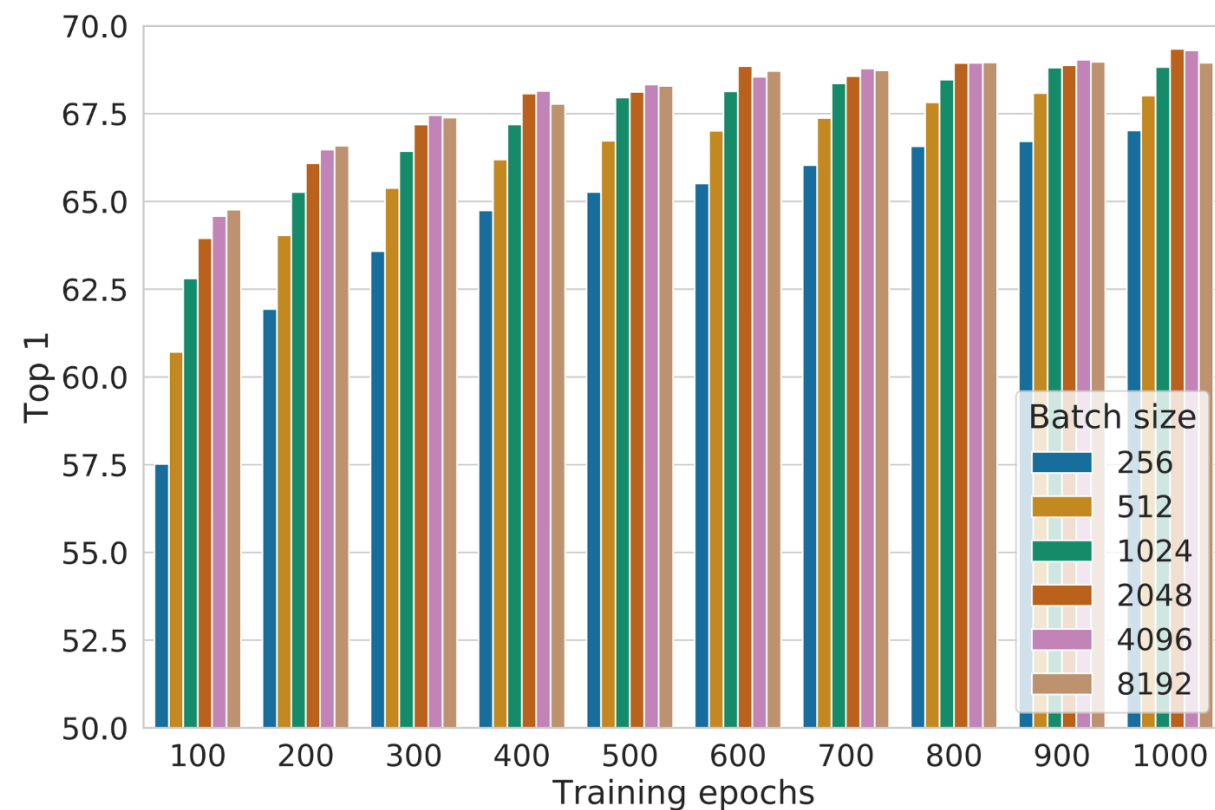
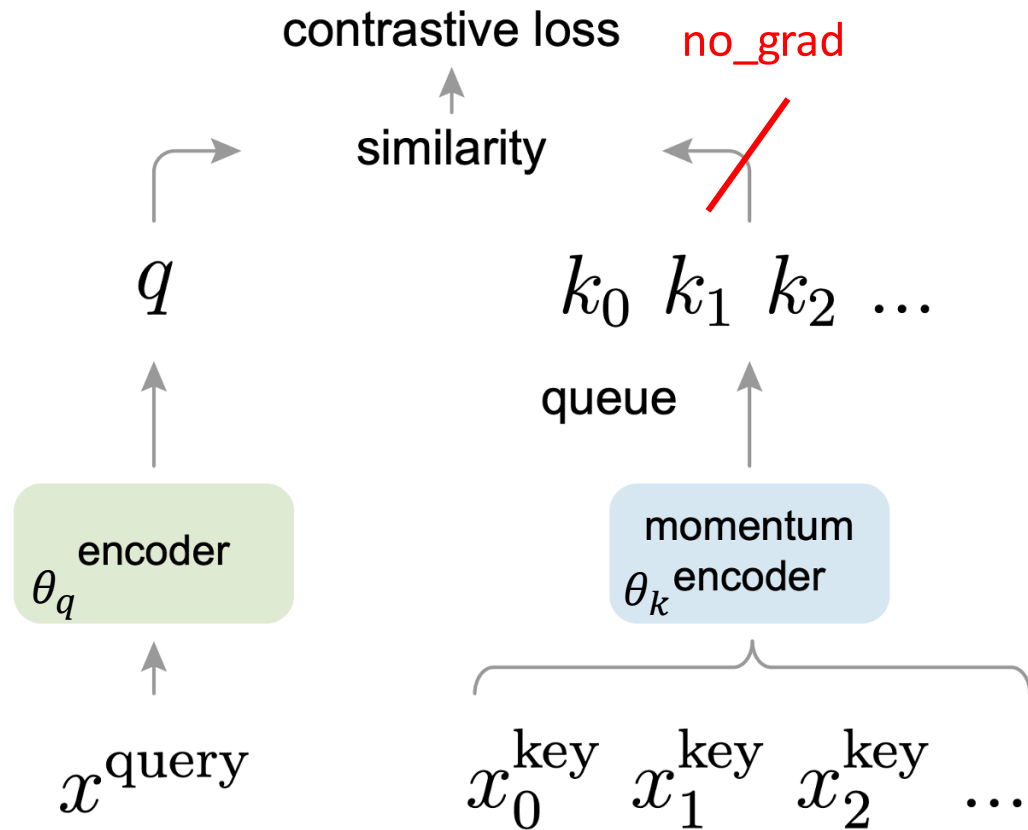


Figure 9. Linear evaluation models (ResNet-50) trained with different batch size and epochs. Each bar is a single run from scratch.¹⁰

Momentum Contrastive Learning (MoCo)



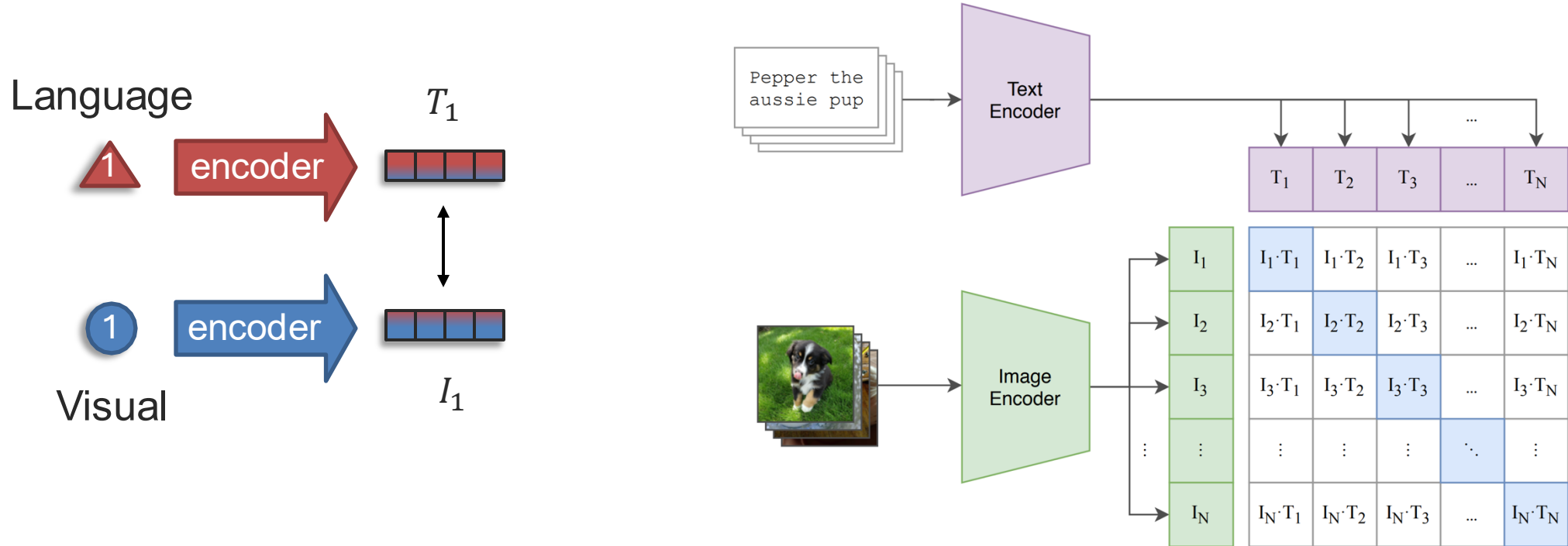
Key differences to SimCLR:

- Keep a running **queue** of keys (negative samples).
- Compute gradients and update the encoder **only through the queries**.
- Decouple min-batch size with the number of keys: can support **a large number of negative samples**.
- The key encoder is **slowly progressing** through the momentum update rules:

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$$

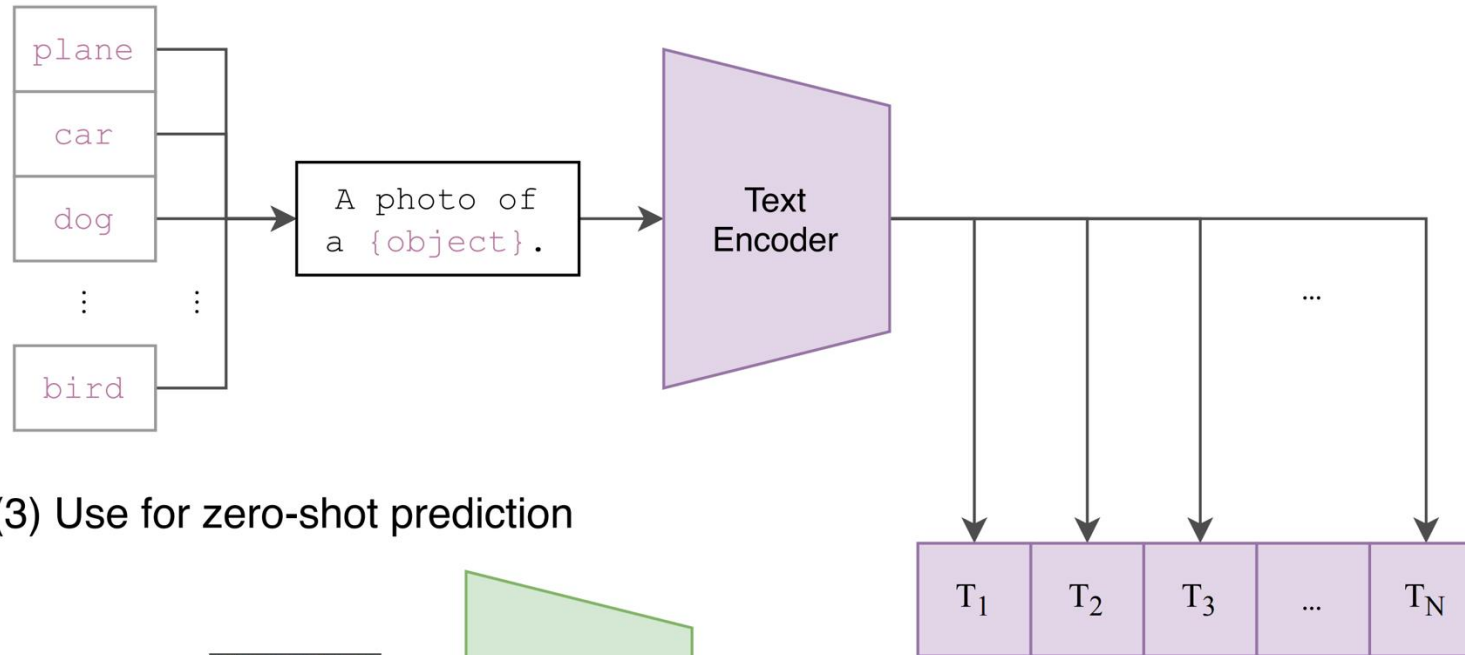
Contrastive Language–Image Pre-training

- Dataset of 400 million (image, text) pairs collected from the internet

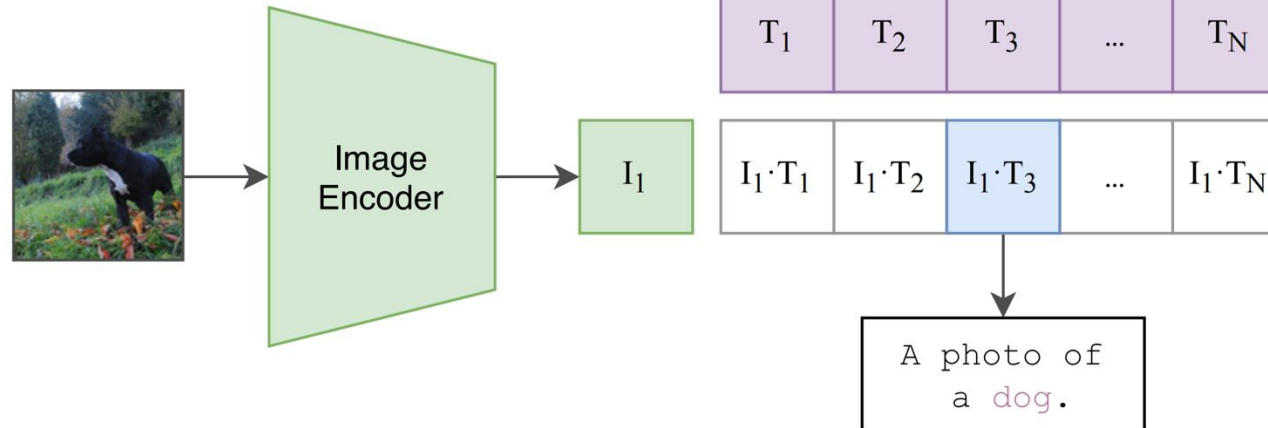


Zero-shot classification

(2) Create dataset classifier from label text

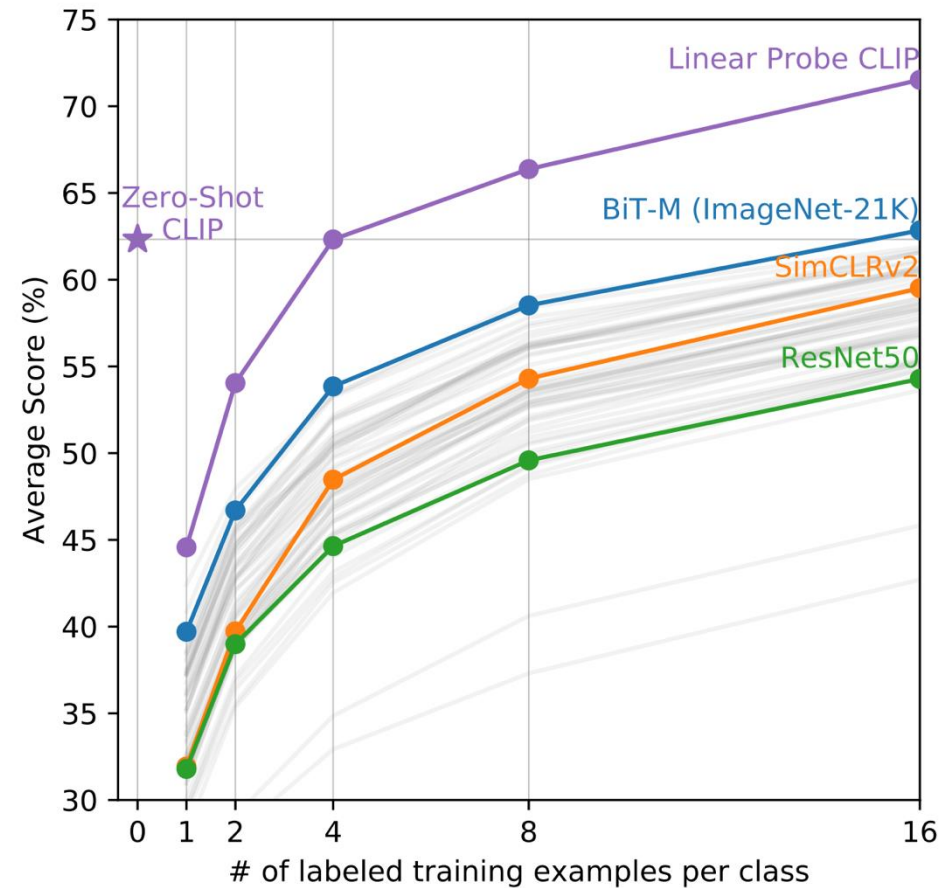


(3) Use for zero-shot prediction

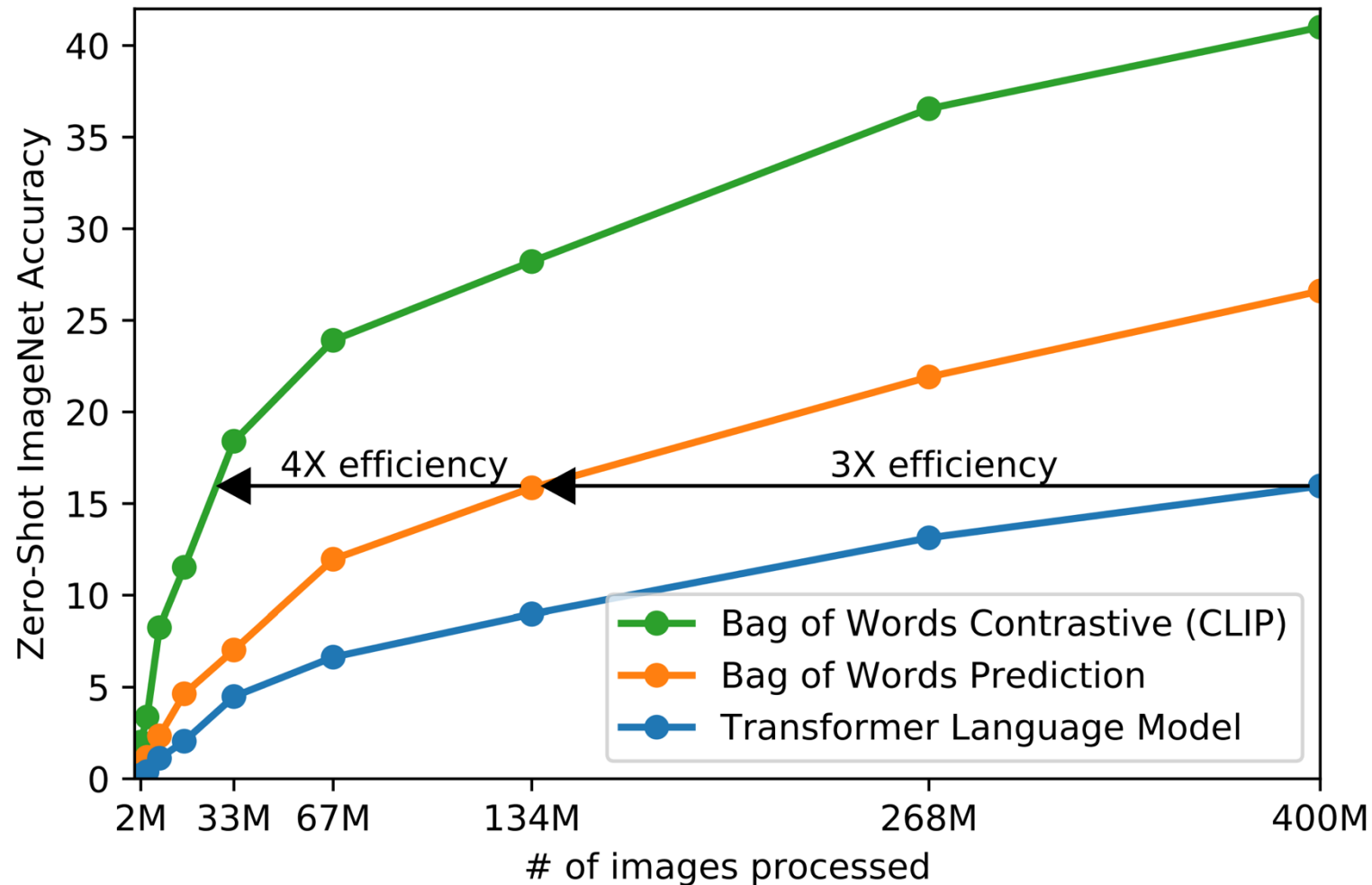


Great representations for transfer learning

- CLIP representations surpass previous on few-shot transfer learning
- Zero-shot CLIP outperforms 2-shot linear probes



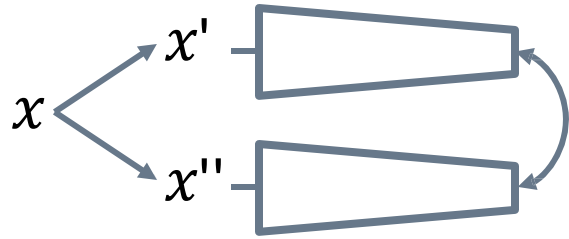
Contrastive learning efficiently learns representations



Previously:

Self-Supervised Paradigms in Vision

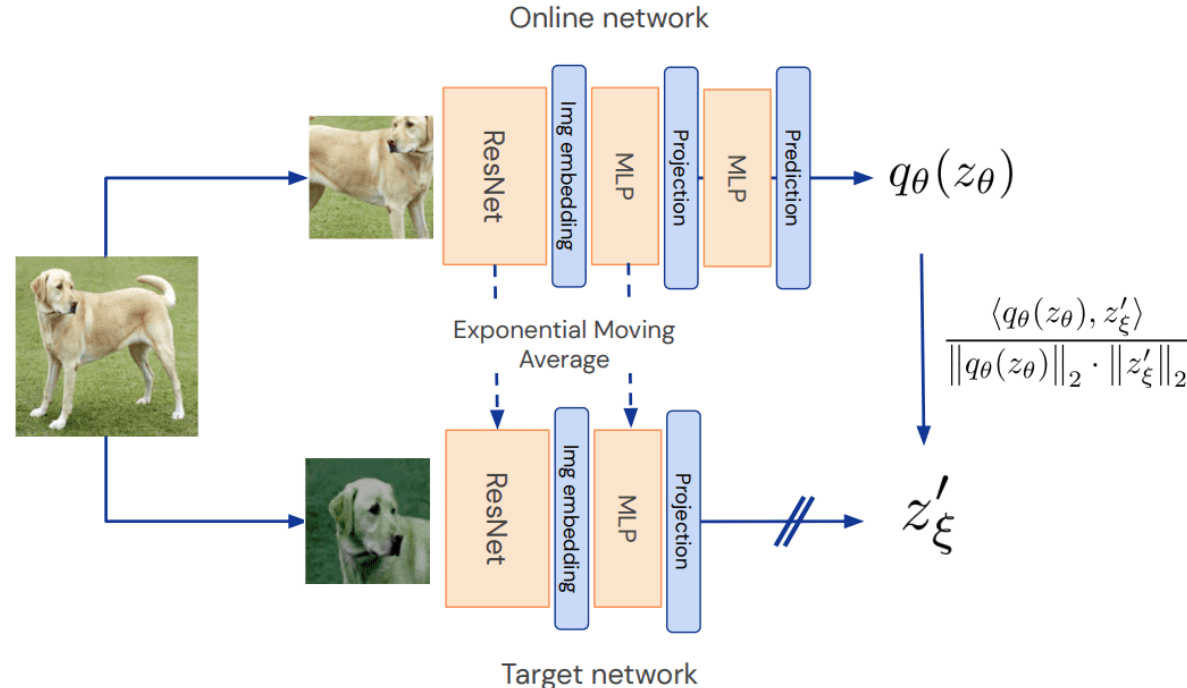
- Contrastive / Siamese



- Compare data points in the latent *representation* space
- Computer vision: SimCLR, MoCo, ... with *augmentations*

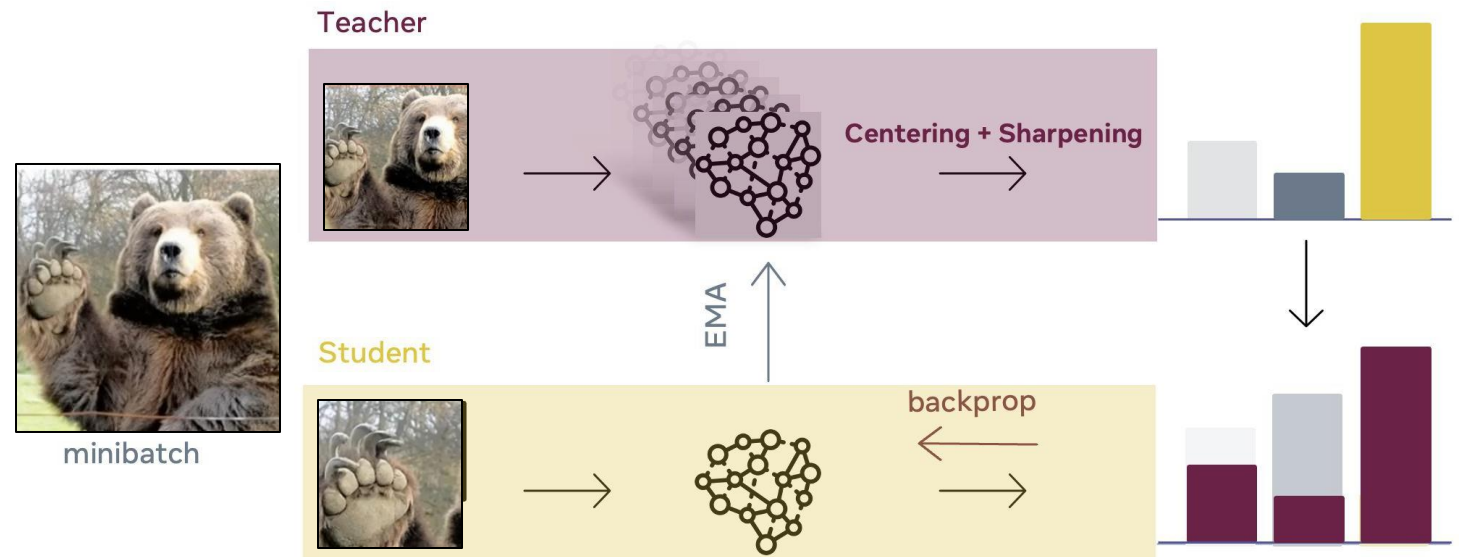
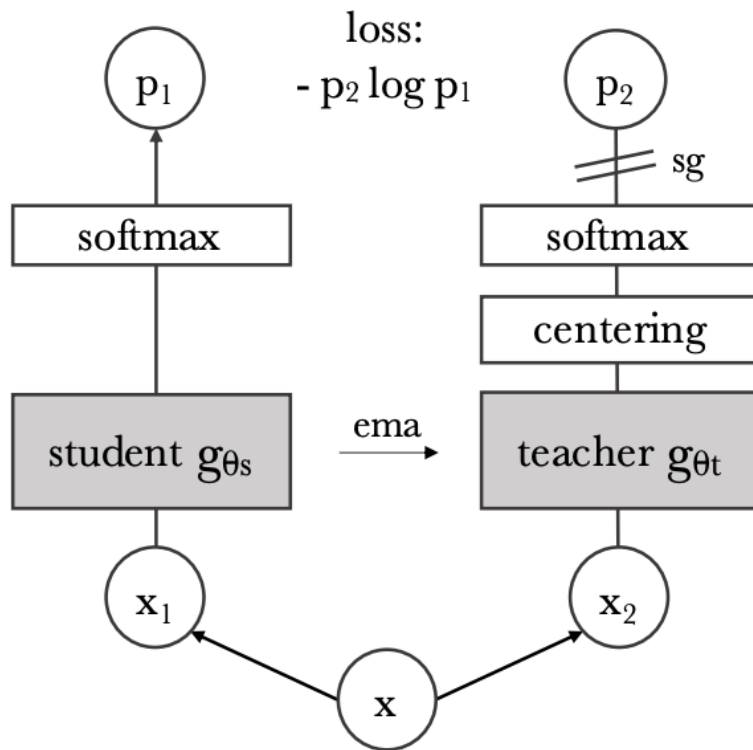
BYOL (Build Your Own Latents)

- Learns by predicting the representation of another network (a *target* network).
- works **without negative samples** and **without contrastive loss**.



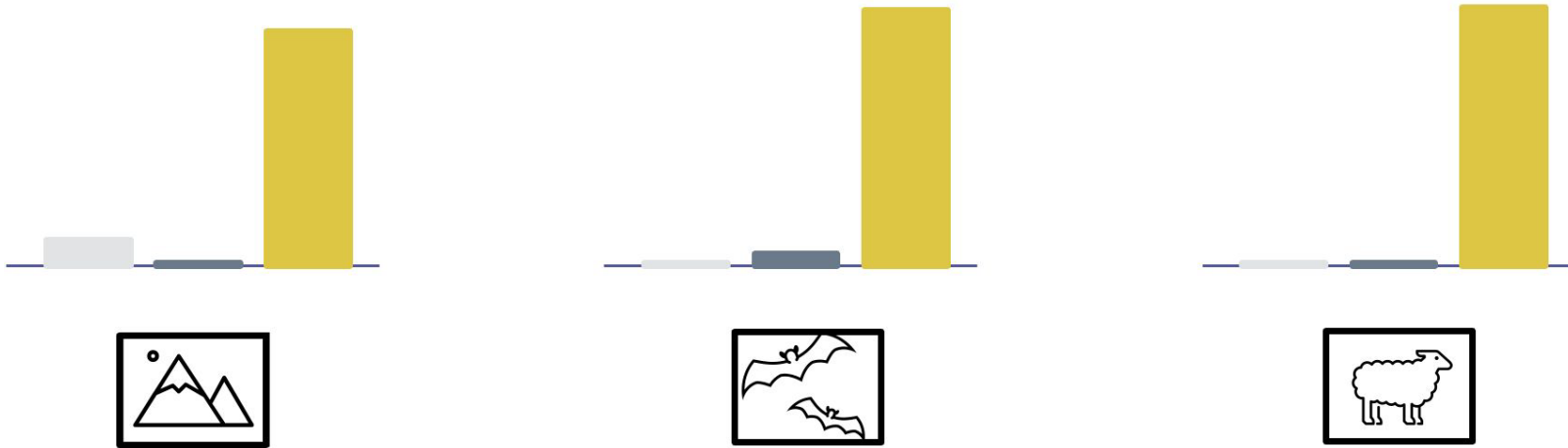
DINO: “Self” Distillation

- DINO learns by matching a “student” logits to a “teacher” logits

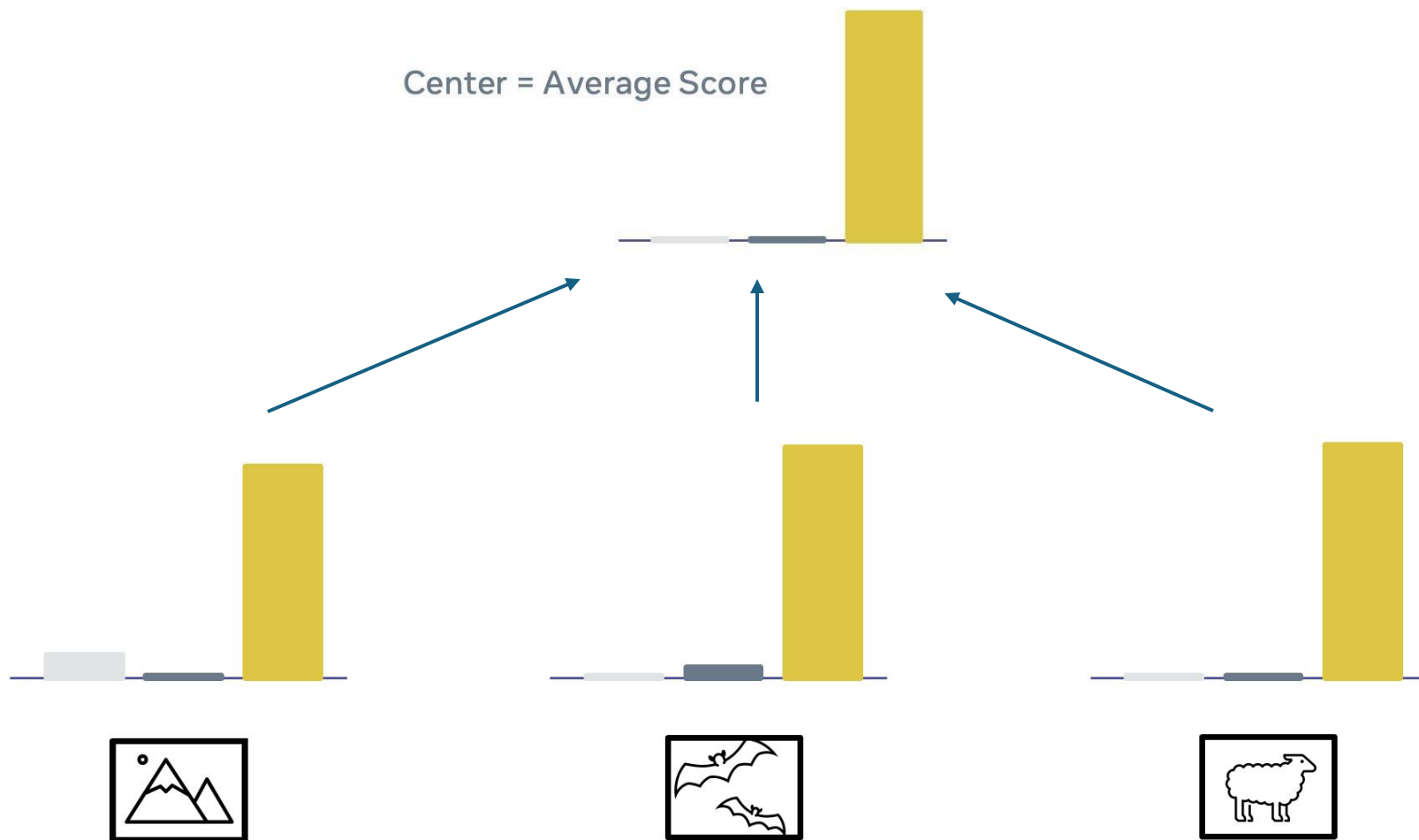


$$\theta_{\text{teacher}} \leftarrow \tau \cdot \theta_{\text{teacher}} + (1 - \tau) \cdot \theta_{\text{student}}$$

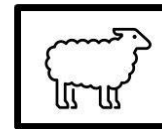
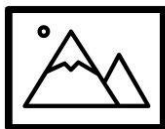
Collapse to constant output



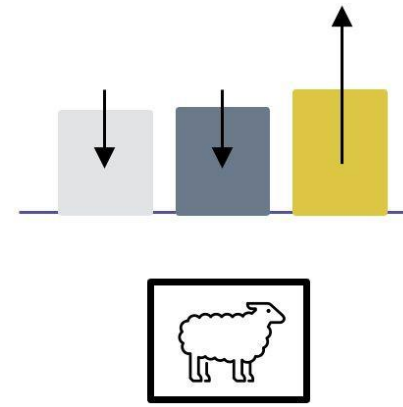
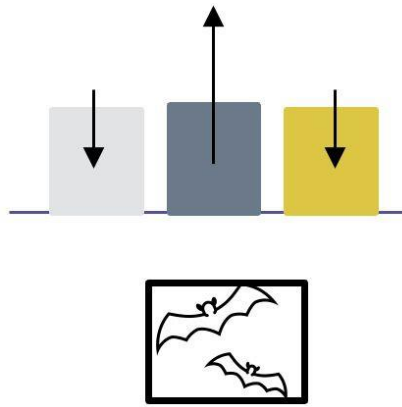
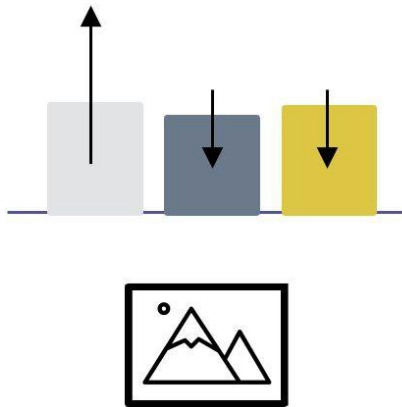
Centering



Subtract average score



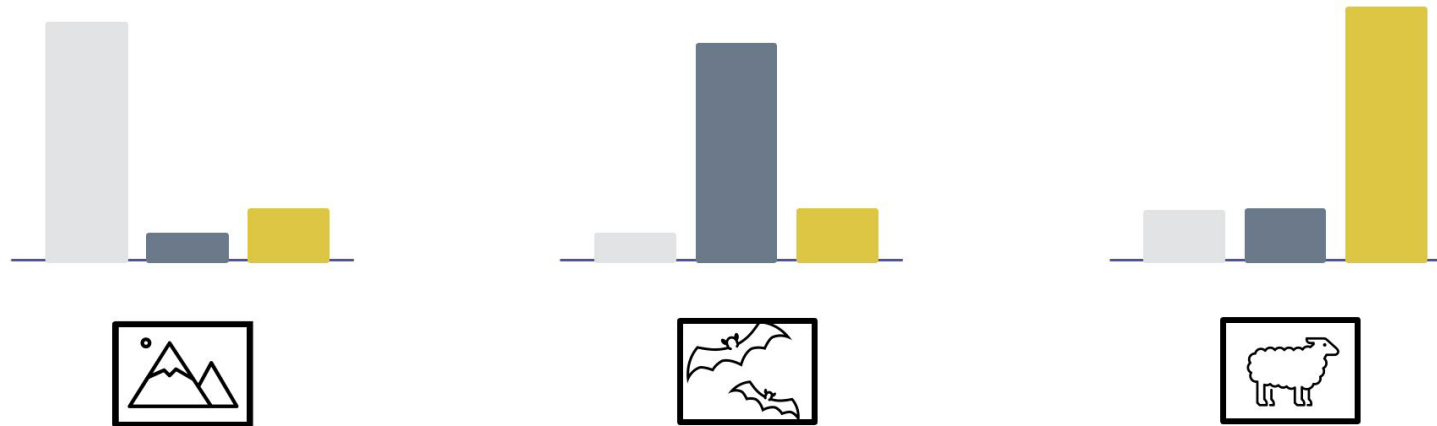
Now it collapse to uniform logits



Sharpening (softmax + temperature)

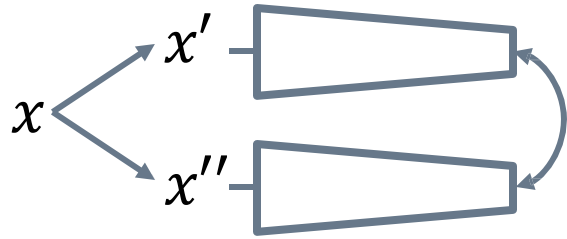
$$p = \text{softmax}\left(\frac{z}{\tau}\right)$$

- Small temperature $\rightarrow$ distribution becomes *sharper* (more peaked)

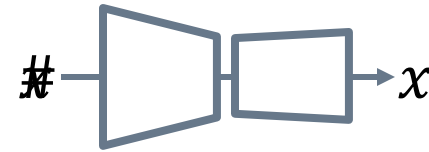


Self-Supervised Paradigms in Vision

- Contrastive / Siamese

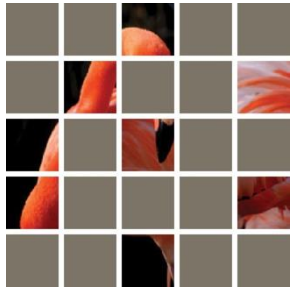


- Reconstructive / Auto-Encoding



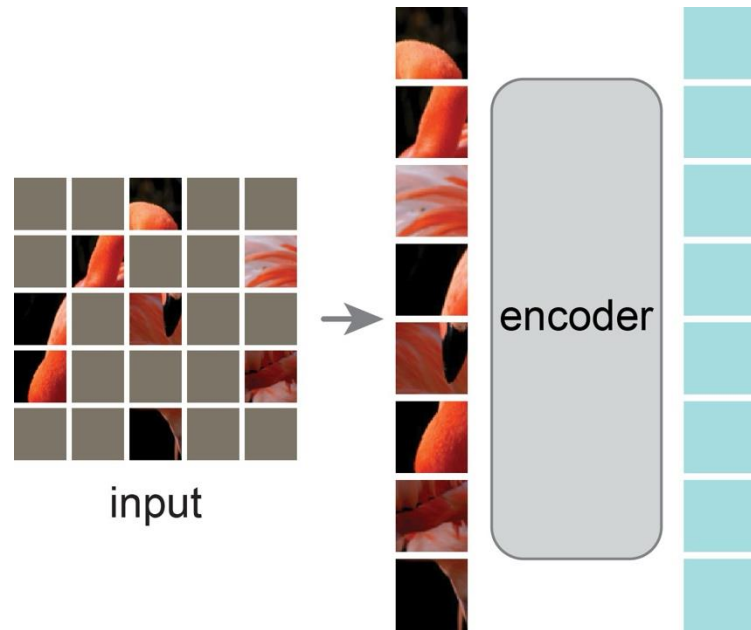
- Reconstruct *corrupted* data points
- *Grounded* in the input space

How MAE Works?



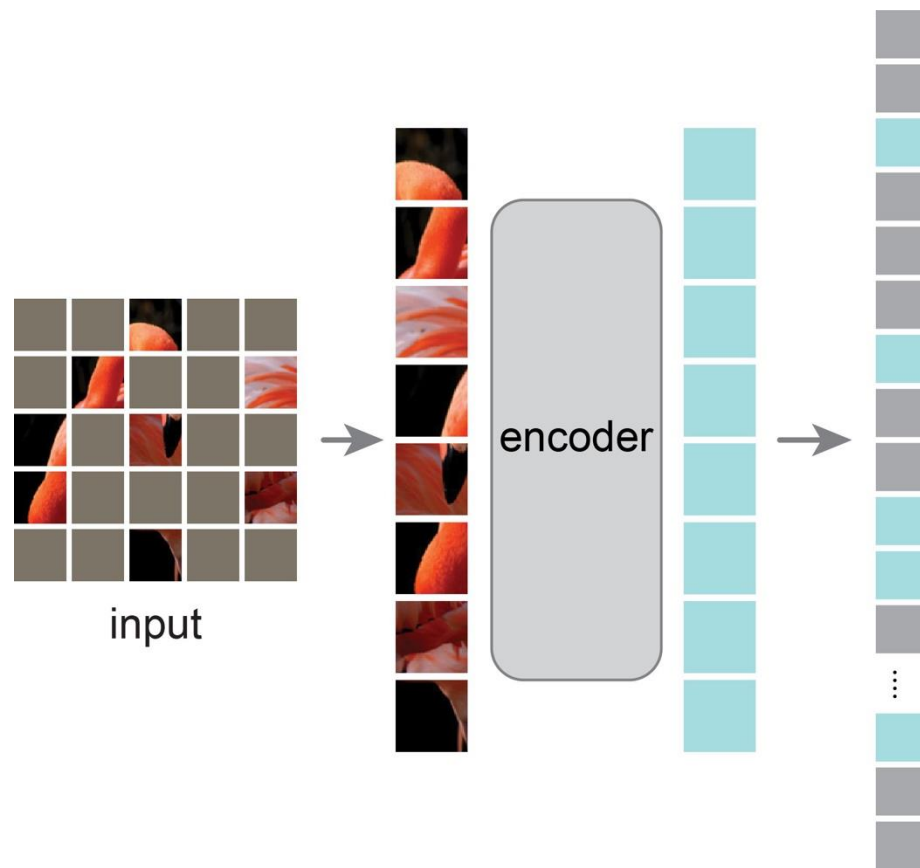
Random masking

How MAE Works?



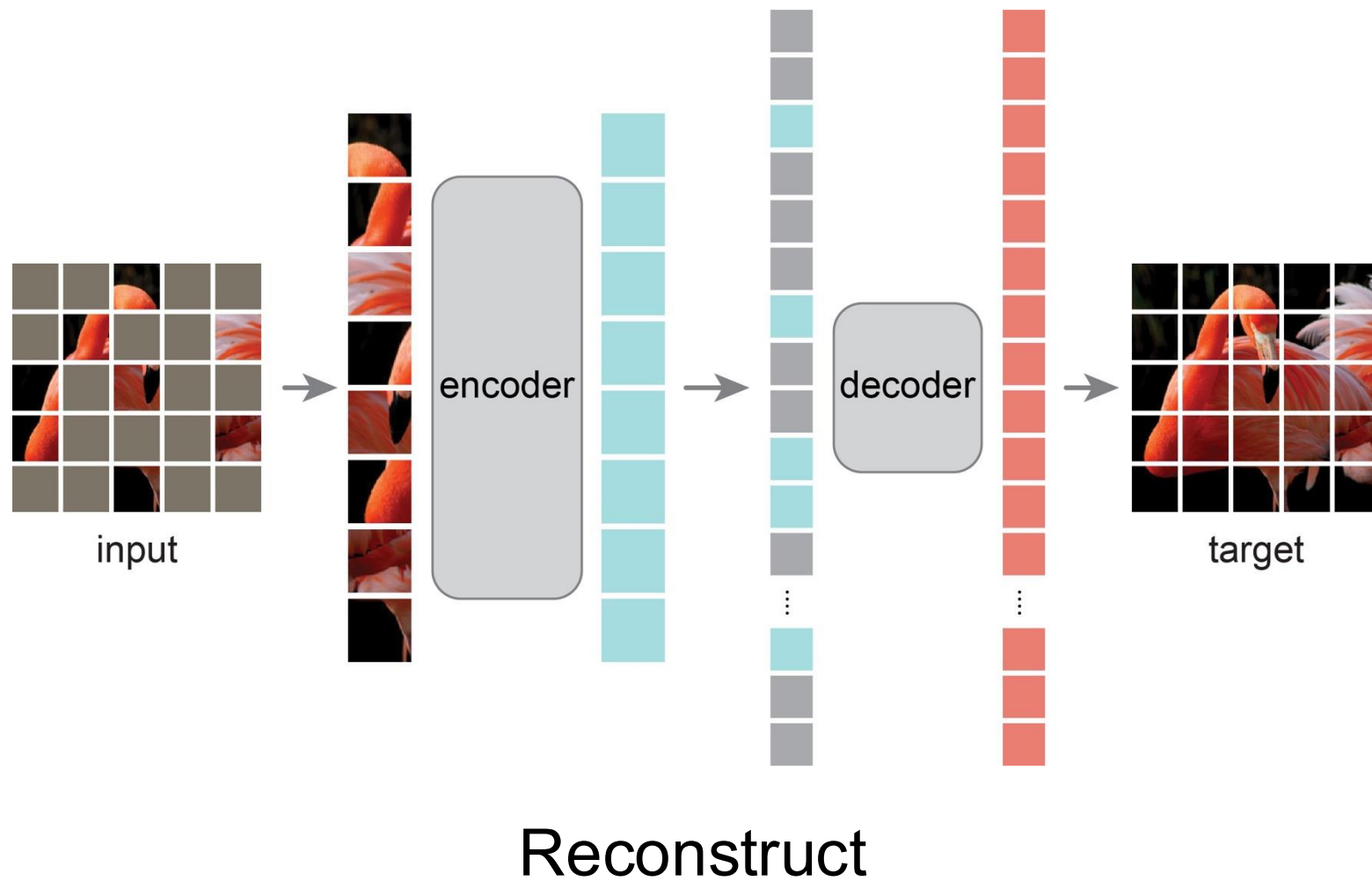
Encode visible patches

How MAE Works?



Add mask tokens

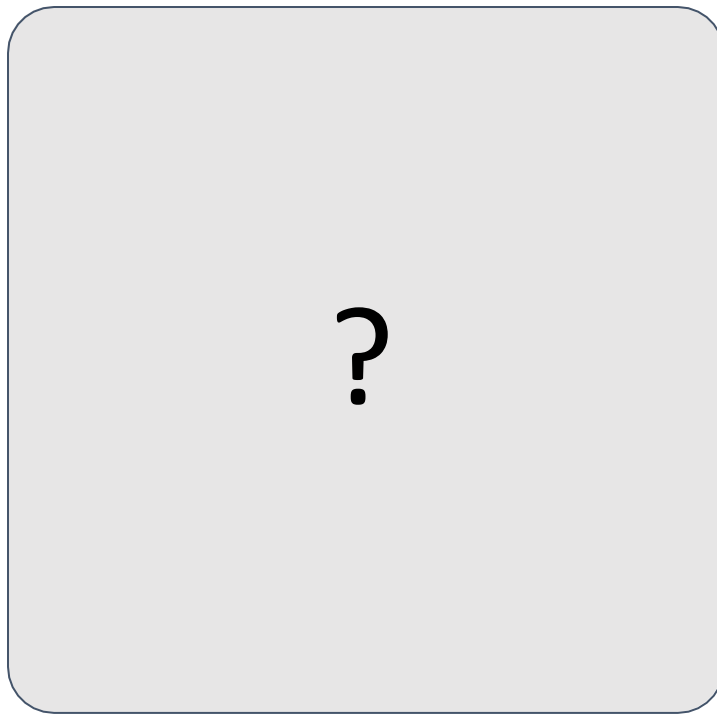
How MAE Works?



MAE Reconstruction Example



Masked input: 80%

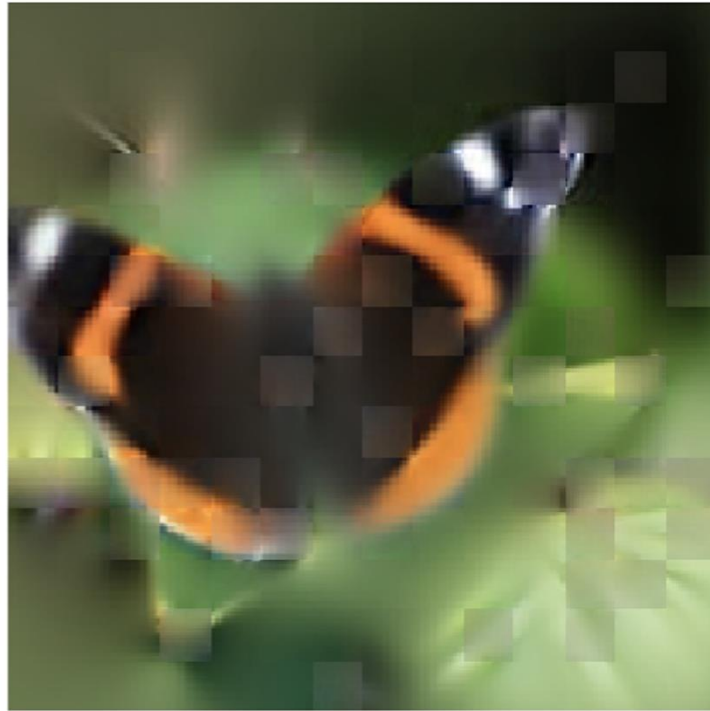


You guess?

MAE Reconstruction Example



Masked input: 80%

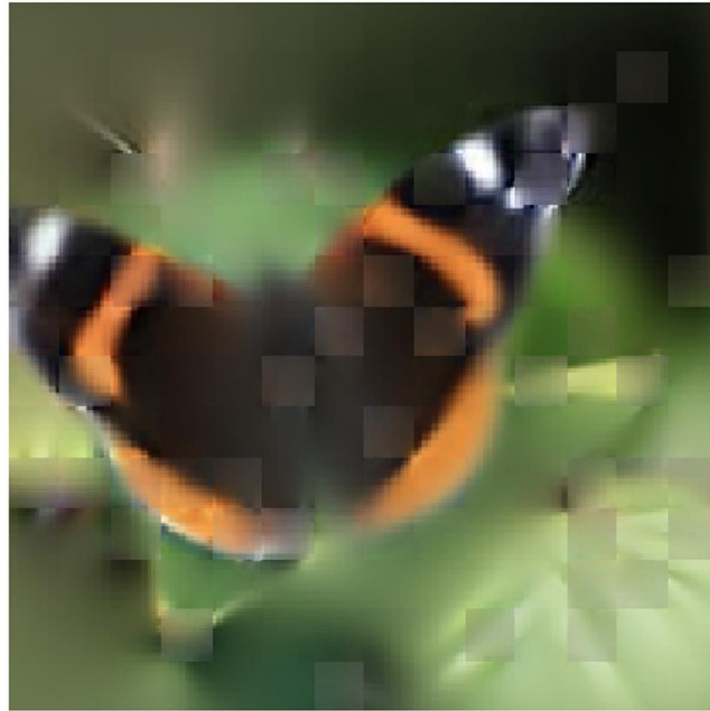


MAE's guess

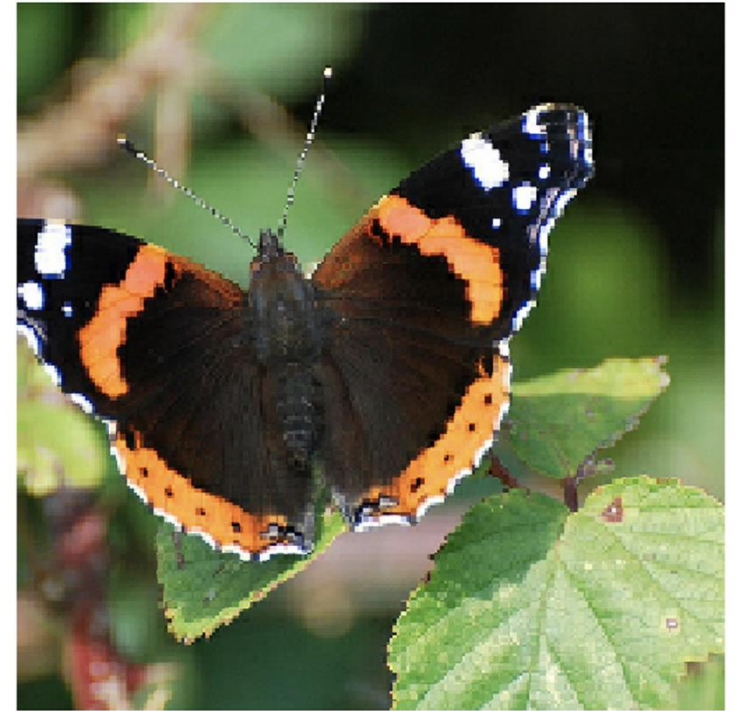
MAE Reconstruction Example



Masked input: 80%

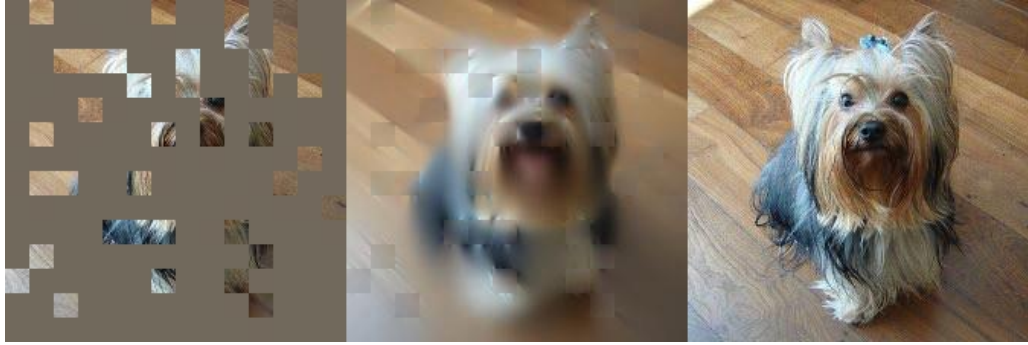
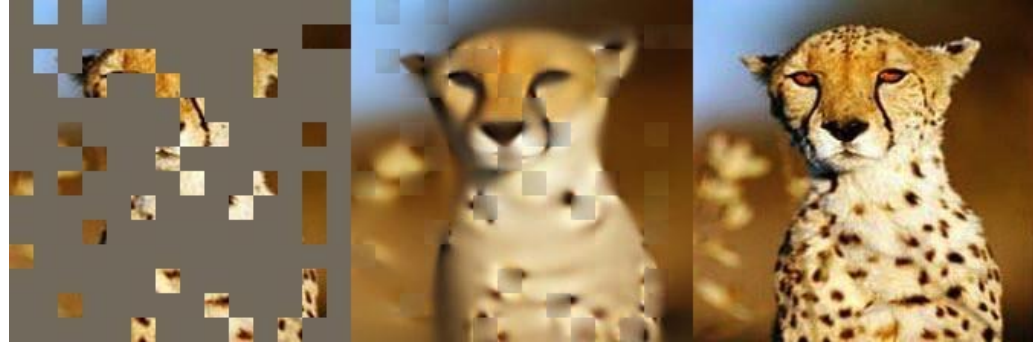


MAE's guess

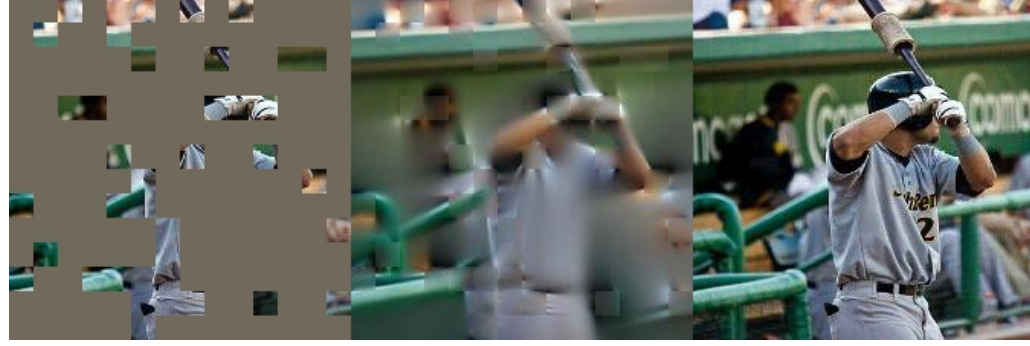
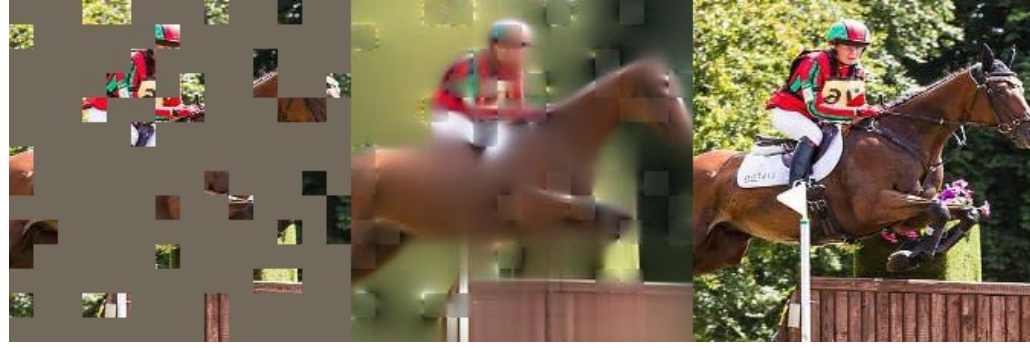


Ground truth

ImageNet val set (unseen)



COCO val set (unseen)





original



75% mask



85% mask

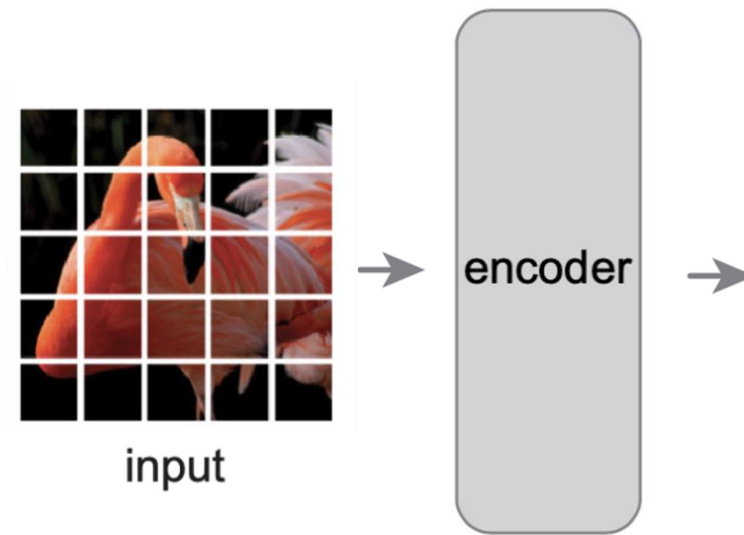


95% mask

MAE Can Generalize

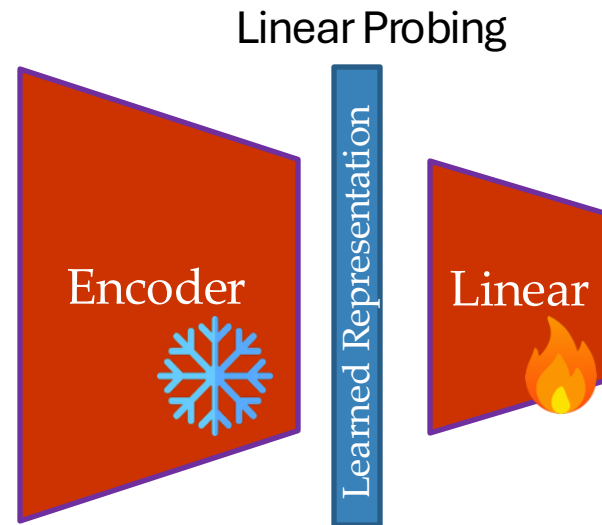
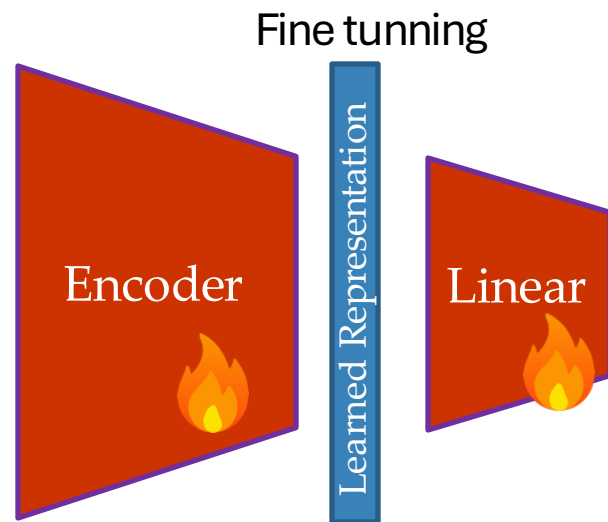
Encoder Only for Downstream Tasks

- After MAE pre-training, just *throw away* the decoder
- Encoder is used for representations with *full-sequence* input

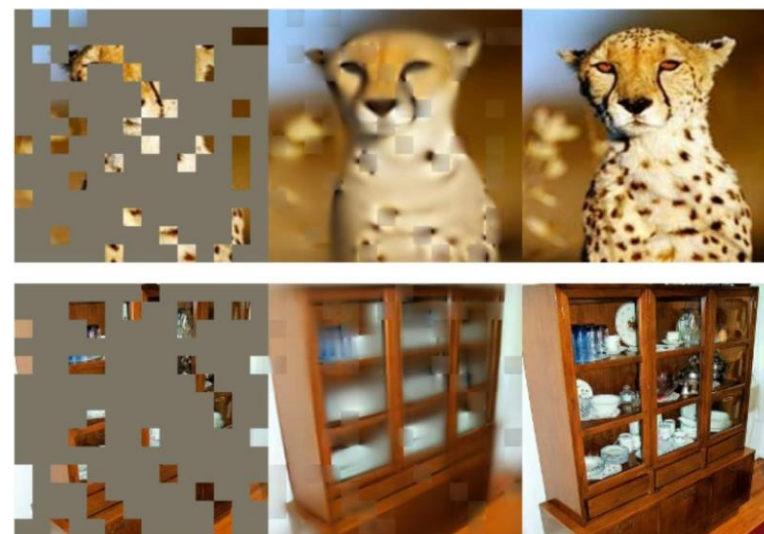
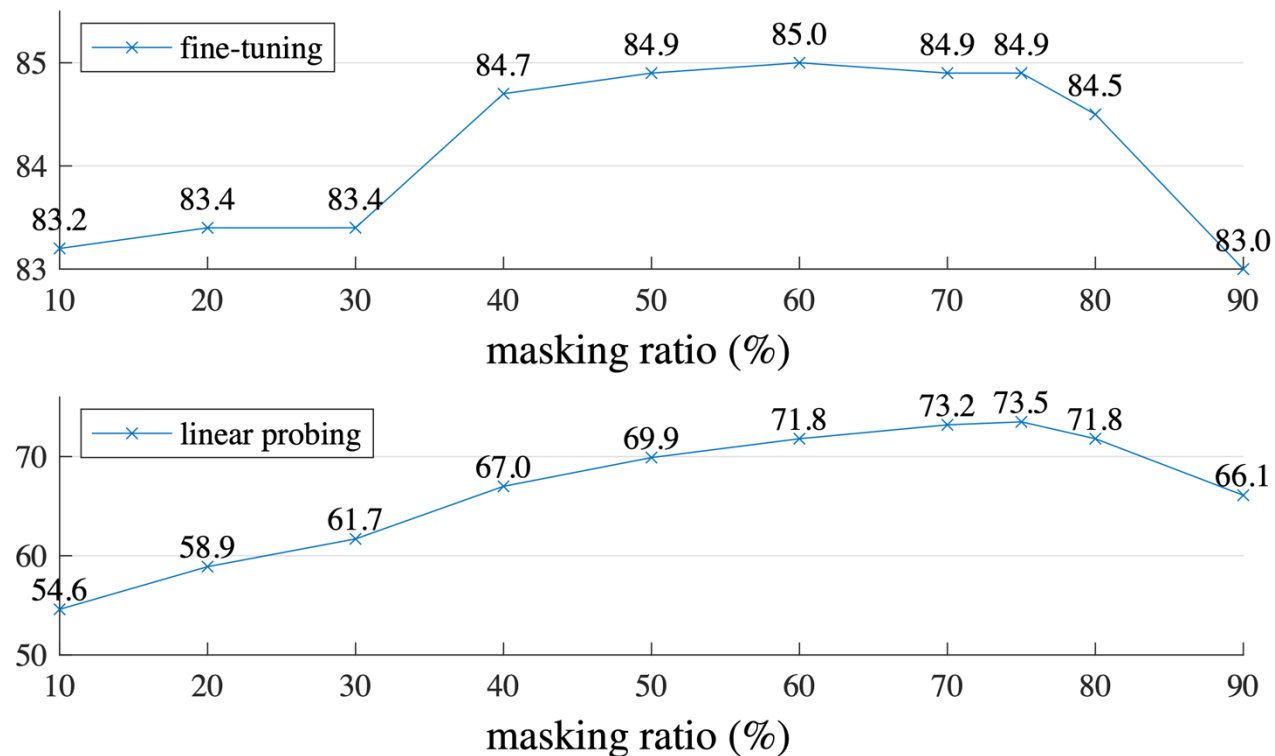


Linear Probing

- In linear probing, the pre-trained model is fixed, and only one linear layer is added
- linear probing: provides a measure of representation quality of a pre-training in restricted conditions



Masked Auto-Encoder (MAE)



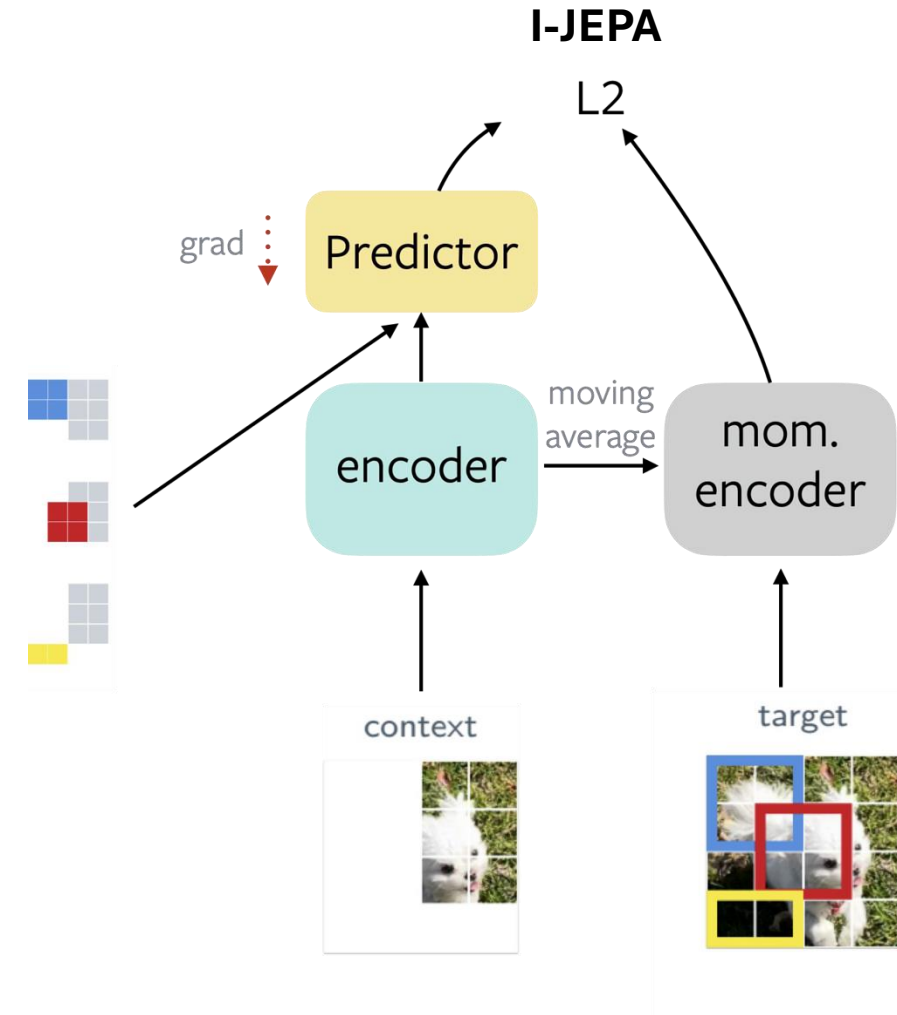
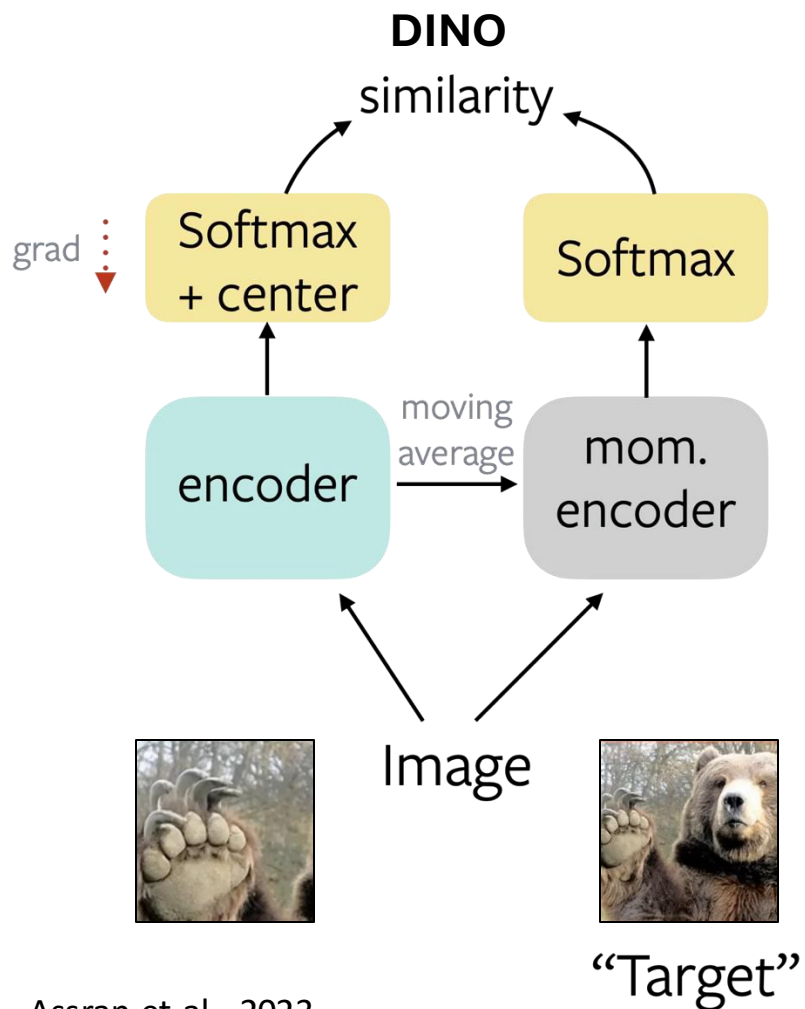
Results

method	pre-train data	ViT-B	ViT-L	ViT-H	ViT-H ₄₄₈
scratch, our impl.	-	82.3	82.6	83.1	-
DINO [5]	IN1K	82.8	-	-	-
MoCo v3 [9]	IN1K	83.2	84.1	-	-
BEiT [2]	IN1K+DALLE	83.2	85.2	-	-
MAE	IN1K	<u>83.6</u>	<u>85.9</u>	<u>86.9</u>	87.8

- We'll talk about BEiT and ViT after we learn about Transformer
- We'll learn about DINO next

I-JEPA

I-JEPA: model predicts masked-region **latents** (no softmax targets) conditioned on visible context.



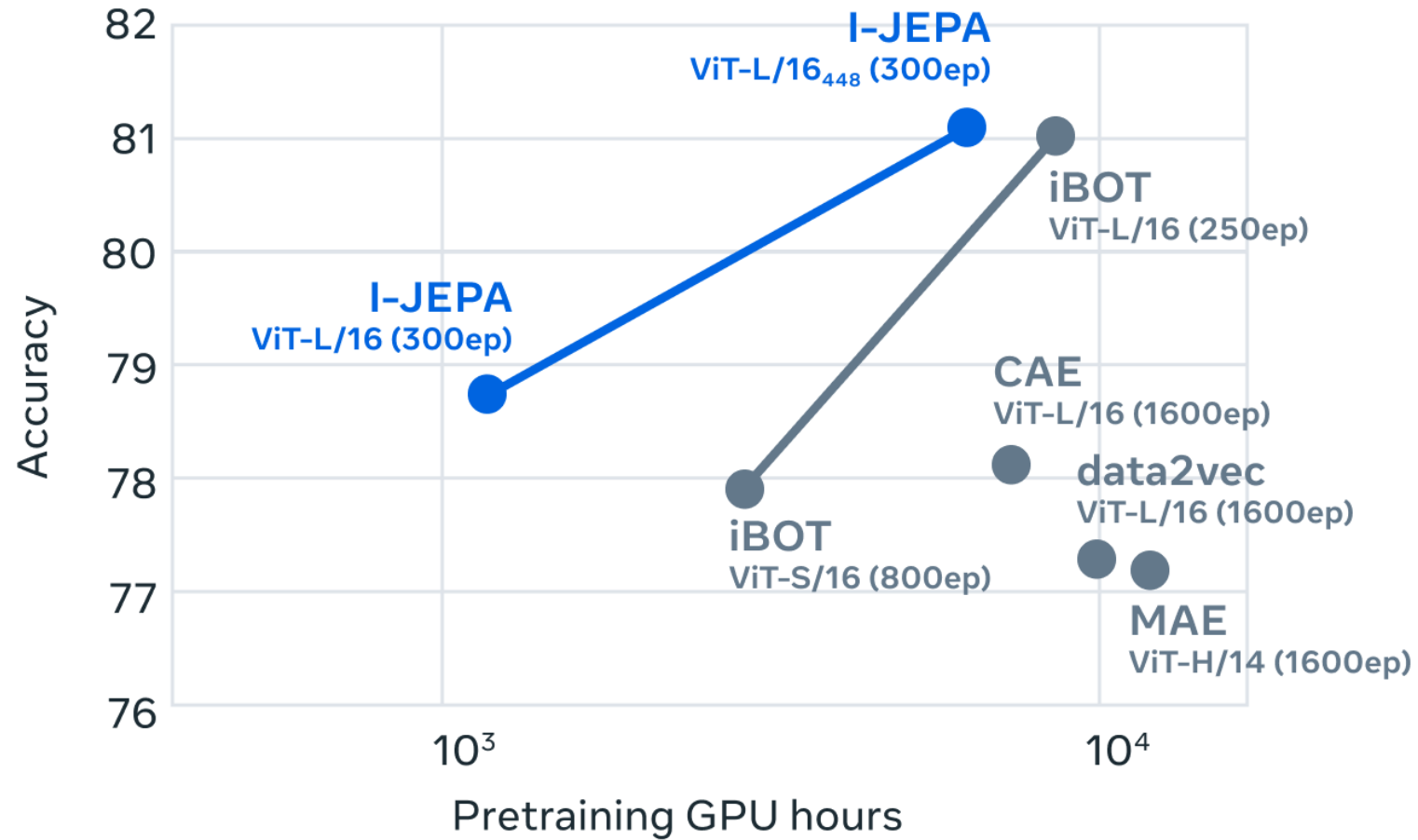
I-JEPA representations

- Model reads context to predict missing **latents**.
- Predictions can be used for image-masks encoding
- Example use case:



Representation converted into sketch.

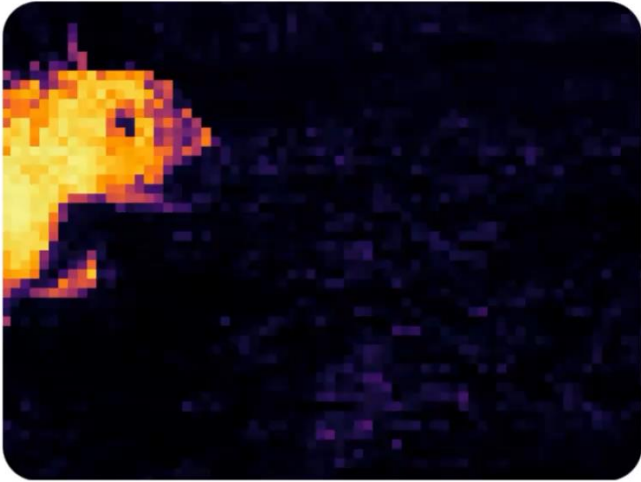
ImageNet-1K linear evaluation



Summarize

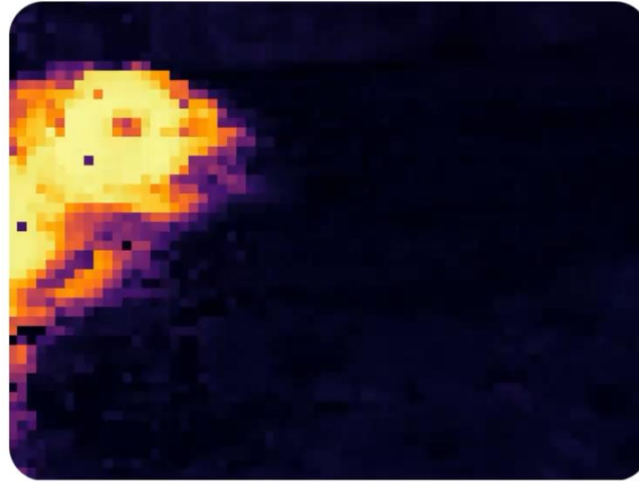
- **Contrastive:** SimCLR, CLIP.
- **Generative / Reconstruction:** MAE.
- **Self-distillation:** DINO.
- **Predictive / Conditional:** I-JEPA.

Current SOTA – DINOv3



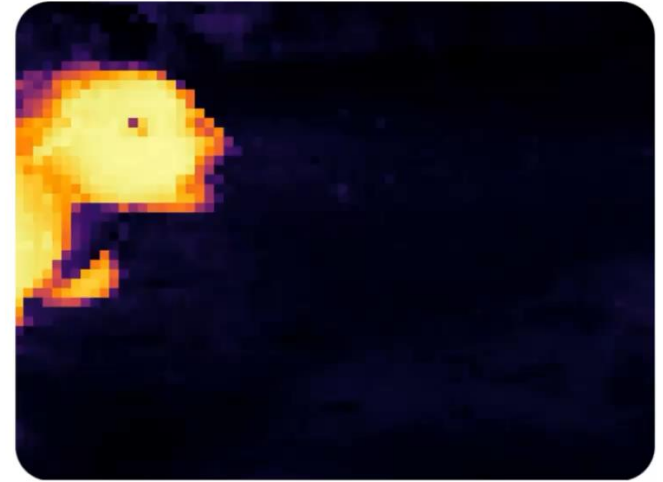
DINO

Initial research proof-of-concept, with 80M-parameter models trained on 1M images.



DINOv2

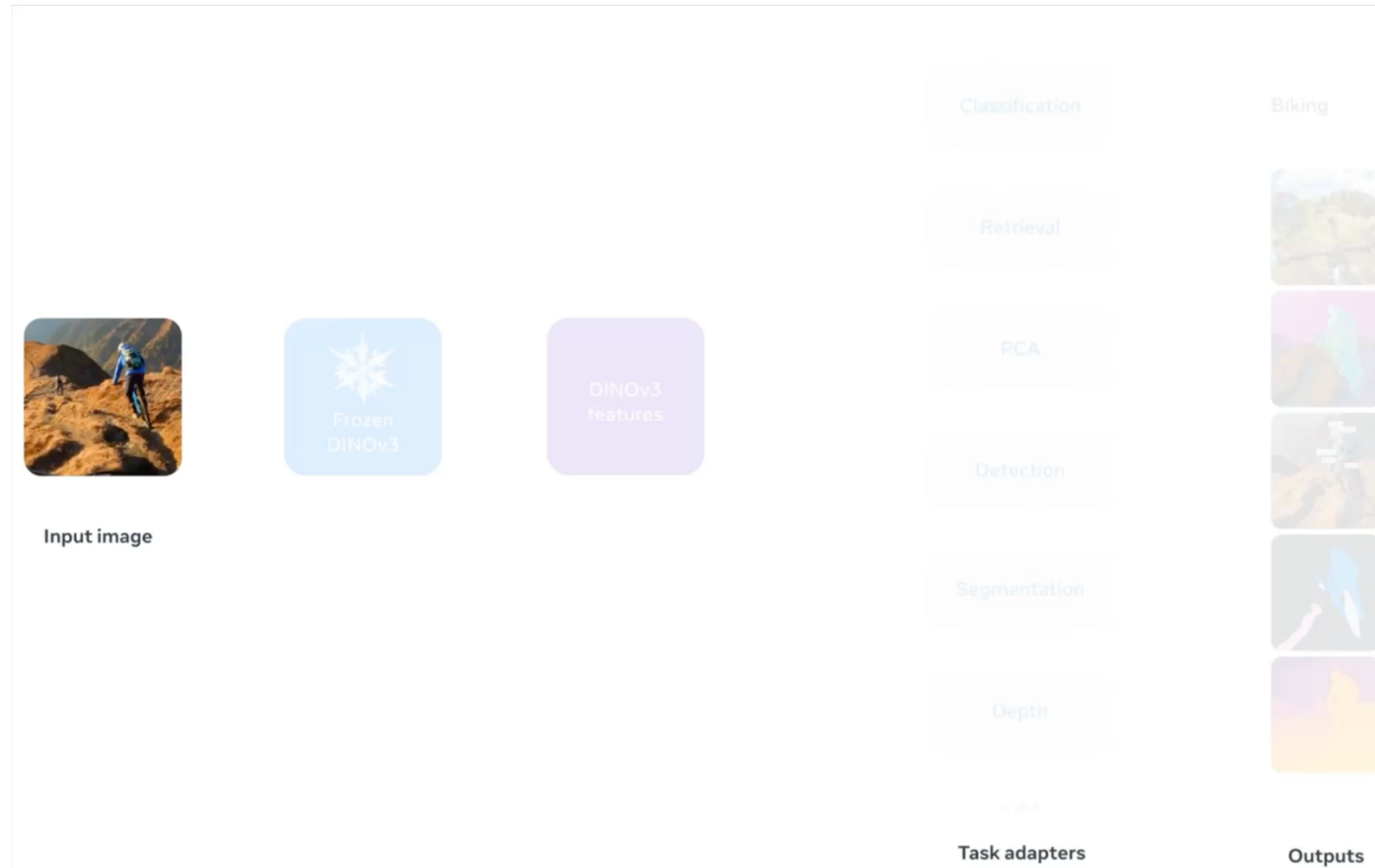
First successful scaling of a SSL algorithm. 1B-parameter models trained on 142M images.



DINOv3

An order of magnitude larger training compared to v2, with particular focus on dense features.

Useful for all core vision tasks



Results

TASK	BENCHMARK	DINO ViT-B/8 0.09B	DINOv2 ViT-G/14 1.1B	DINOv3 ViT-7B/16 7B	SIGLIP 2 ViT-G-OPT/16 1.8B	PE ViT-G/14 1.9B
Segmentation	ADE-20k	31.8	49.5	55.9	42.7	38.9
Depth estimation	NYU ↓	0.537	0.372	0.309	0.494	0.436
Video tracking	DAVIS	68.7	76.6	83.3	62.9	49.8
Instance retrieval	Met	17.1	44.6	55.4	13.9	10.6
Image classification	ImageNet ReaL	85.9	89.9	90.4	90.5	90.4
Image classification	ObjectNet	39.9	66.4	79.0	78.6	80.2
Fine-grained Image classification	iNaturalist 2021	68.3	86.1	89.8	82.7	87.0

Next, sequence learning

Appendix: LeJEPA:Lean Joint-Embedding Predictive Architecture

- Proposes the isotropic Gaussian as the optimal target distribution for JEPA embeddings.
- Uses Sketched Isotropic Gaussian Regularization (SICReg) to push embeddings toward that distribution.
- implementable in about 50 lines of code.
- Shows stable training across hyperparameters, architectures, and datasets.

Appendix: LeJepa Results

Method	Full FT		Frozen	
	1-sh	Full	1-sh	Full
<i>LeJEPa (in-domain)</i>				
ConvNeXt-V2 Nano	29.42	82.72	28.74	76.52
ResNet-34	24.27	83.28	31.08	78.17
<i>Frontier (transfer)</i>				
DINOv2 ViT-S/16	21.05	78.34	27.68	67.62
DINOv3 ViT-S/16	24.71	81.60	30.17	71.38

